

AT32 PMSM FOC Hall Demo (E-Bike-Scooter)

Introduction

This application note describes the SOP (Standard Operating Procedure) for quickly using a motor development board to drive and debug PMSM (Permanent Magnet Synchronous Motor) motors with Hall sensors, applicable to e-bikes, electric scooters, electric motorcycles, and similar applications. ArteryTek has a variety of motor development boards, and this document uses the AT-MOTOR-EVB as an example to illustrate the process.

Applicable products:

Product series	AT32F402/405 series
	AT32F403A/407 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32F423 series
	AT32F435/437 series
	AT32F45x series
	AT32L021 series
	AT32M412/M416 series

Contents

1	Motor control library algorithms	7
2	Environment requirements	8
	2.1 Hardware requirements	8
	2.2 Software requirements	10
3	Quick start guide	14
	3.1 Modify motor control mode and parameters	16
	3.2 Connection settings	21
	3.3 Open loop control	22
	3.4 Hall phase sequence self-learning.....	22
	3.5 Voltage control.....	23
	3.6 Auto identification and tuning of parameters.....	24
	3.7 ID tune.....	27
	3.8 IQ tune	29
	3.9 Current loop control.....	30
	3.10 Speed loop control	32
	3.11 Position loop control	34
	3.12 Flux-Weakening Control (Surface-Mounted Permanent Magnet Synchronous Motor, SPMSM).....	35
	3.13 MTPA/MTPV Control (Interior Permanent Magnet Synchronous Motor, IPMSM) ...	36
	3.14 Reverse mode	40
	3.15 Brake recharging mode	40
	3.16 Brake switch	41
	3.17 Anti-Theft mode.....	41
	3.18 Parking mode	42
	3.19 External voltage control/software control mode	44
4	Revision history.....	46

List of tables

Table 1. Flash memory size and ROM configuration	10
Table 2. Macro definition setting (internal crystal)	16
Table 3. Macro definition setting (EVB version).....	16
Table 4. Macro definition setting (lower-side MOS inverted output).....	16
Table 5. Macro definition setting (vector control mode).....	16
Table 6. Macro definition setting (current sampling method)	16
Table 7. Macro definition setting (two-shunt current sampling method).....	17
Table 8. Macro definition setting (MOSFET's $R_{DS(on)}$)	17
Table 9. Macro definition setting (dual ADCs current sampling)	17
Table 10. Macro definition setting (hall sensor).....	17
Table 11. Macro definition setting (D/Q-axis current low-pass filter).....	17
Table 12. Macro definition setting (a/b/c phase current low-pass filter)	17
Table 13. Macro definition setting (E-Bike-Scooter project)	18
Table 14. Macro definition setting (bus current limit).....	18
Table 15. Macro definition setting (MOS $R_{DS(on)}$ auto calibration)	18
Table 16. Macro definition setting (motor type)	18
Table 17. Macro definition setting (MTPA/MTPV lookup generation).....	18
Table 18. Macro definition setting (MTPA/MTPV lookup table-based torque control)	19
Table 19. Macro definition setting (field weakening control).....	19
Table 20. Macro definition setting (current decouple control).....	19
Table 21. Macro definition setting (motor winding parameter identification).....	19
Table 22. Macro definition setting (Artery motor UI tool).....	19
Table 23. Motor parameter definition	20
Table 24. Document revision history.....	46

List of figures

Figure 1. AT-MOTOR-EVB V1.x	8
Figure 2. AT-MOTOR-EVB V2.x	9
Figure 3. AT32F413RCT7 ROM configuration (Keil).....	11
Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)	12
Figure 5. AT32F413RCT7 ROM configuration (IAR).....	13
Figure 6. Connection settings	21
Figure 7. Select open loop control mode.....	22
Figure 8. Open loop control parameters.....	22
Figure 9. Hall phase sequence self-learning	22
Figure 10. Hall phase sequence self-learning open loop voltage	23
Figure 11. Hall phase sequence self-learning macro definition	23
Figure 12. Select Voltage control mode.....	23
Figure 13. Voltage control parameters	24
Figure 14. Nominal current macro definition	24
Figure 15. Identify winding parameters	25
Figure 16. Motor winding parameters macro definition	25
Figure 17. Nominal voltage.....	25
Figure 18. Auto tune current PI parameters	26
Figure 19. Macro definition of current PI parameters.....	26
Figure 20. Step current diagram	27
Figure 21. Select ID Tune mode.....	27
Figure 22. PID and step current parameters	27
Figure 23. Diagram parameter settings (ID tune).....	28
Figure 24. ID tune waveform	28
Figure 25. D-axis current PI control parameter macro definition	28
Figure 26. Select ID Tune mode.....	29
Figure 27. PID and step current parameters	29
Figure 28. Diagram parameter settings (IQ tune).....	29
Figure 29. IQ tune waveform	30
Figure 30. Q-axis current PI control parameter macro definition	30
Figure 31. Set target current.....	30
Figure 32. Diagram parameter settings (current loop debugging)	31
Figure 33. Current loop control waveform	31

Figure 34. Speed response waveforms.....	32
Figure 35. PID parameters, acceleration and deceleration.....	32
Figure 36. Set target speed	33
Figure 37. Diagram parameter setting (speed loop control).....	33
Figure 38. Waveform in speed loop control mode.....	33
Figure 39. FWC PID gains and the maximum negative d-axis current limit	35
Figure 40. Target speed setting (FWC)	35
Figure 41. Diagram parameter settings (speed control).....	36
Figure 42. Waveforms in speed control mode (FWC).....	36
Figure 43. Example table for Iq_table and Id_table	37
Figure 44. Macro definitions for MTPA/MTPV table generation.....	38
Figure 45. Enable the macro definition IPM_MTPA_MTPV_TABLE.....	38
Figure 46. Set the per-unit value for the Id current command	38
Figure 47. Diagram parameter settings (MTPA/MTPV)	39
Figure 48. MTPA/MTPV table indices.....	39
Figure 49. MTPA/MTPV table row and column count	39
Figure 50. Reverse speed/current macro definition	40
Figure 51. Reverse flag configuration.....	40
Figure 52. Brake current macro definition	40
Figure 53. Brake flag set.....	41
Figure 54. Id current during anti-theft	42
Figure 55. Anti-theft flag set.....	42
Figure 56. Id current during parking	43
Figure 57. Parking lock flag set	43
Figure 58. External voltage control – potentiometer	44
Figure 59. Control source definitions (upper: software control, lower: external voltage control).....	44
Figure 60. Speed control mode (external voltage control)	45
Figure 61. Torque control mode (external voltage control).....	45
Figure 62. Set target speed in software control mode	45

1 Motor control library algorithms

This document contains the following motor control algorithms:

Target motor type: Three-phase permanent magnet synchronous motor (BLDC)

Control modes:

- FOC vector control

Three-phase PWM method:

- SVPWM

Phase current sensing methods:

- 3-shunt current sensing
- 2-shunt current sensing
- 1-shunt current sensing and reconstruction method

Rotor position detection mode:

- Hall effect position detection

Sensor-based FOC sinusoidal control mode:

- Voltage vector control
- Torque control (current vector control)
- Speed control
- Field weakening control

Auto tuning functions:

- Hall phase sequence self-learning (for hall sensor only)
- Motor winding parameters identification
- Current PI control parameters self-tuning

E-Bike-Scooter control mode:

- DC current limitation
- MOS's RDS(on) auto calibration

2 Environment requirements

2.1 Hardware requirements

PMSM (BLDC) motor, AT-Link or third-party debugger, and motor control evaluation board (AT-MOTOR-EVB) are required. For details about hardware configurations, please refer to *AT32_LV_Motor_Control_EVB_User_Manual* (UM0011 for EVB_V1.x or UM0014 for EVB_V2.x).

- PMSM (BLDC) motor
- Debugger
- Motor control EVB: AT-MOTOR-EVB V1.x or AT-MOTOR-EVB V2.x

Figure 1. AT-MOTOR-EVB V1.x

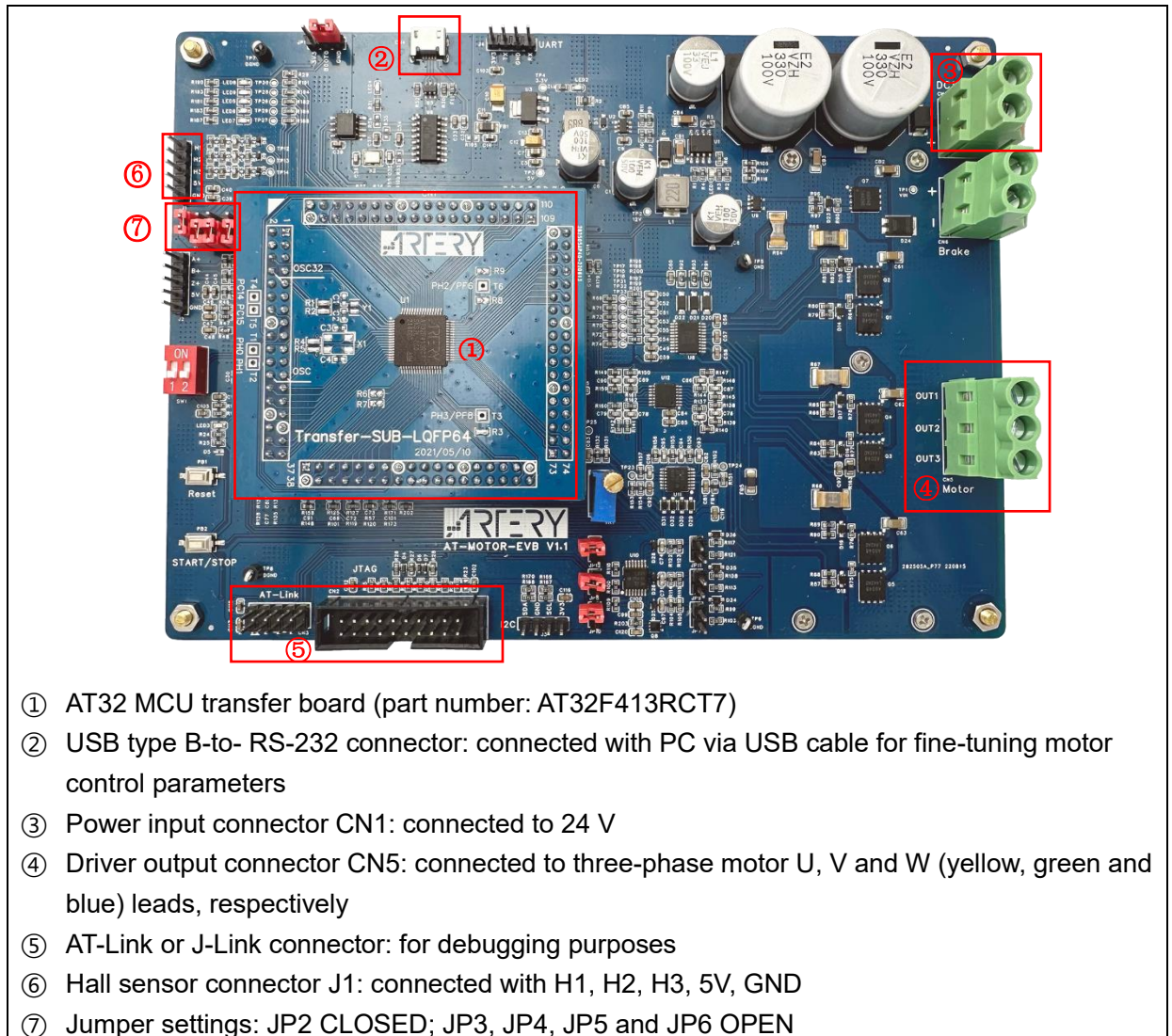
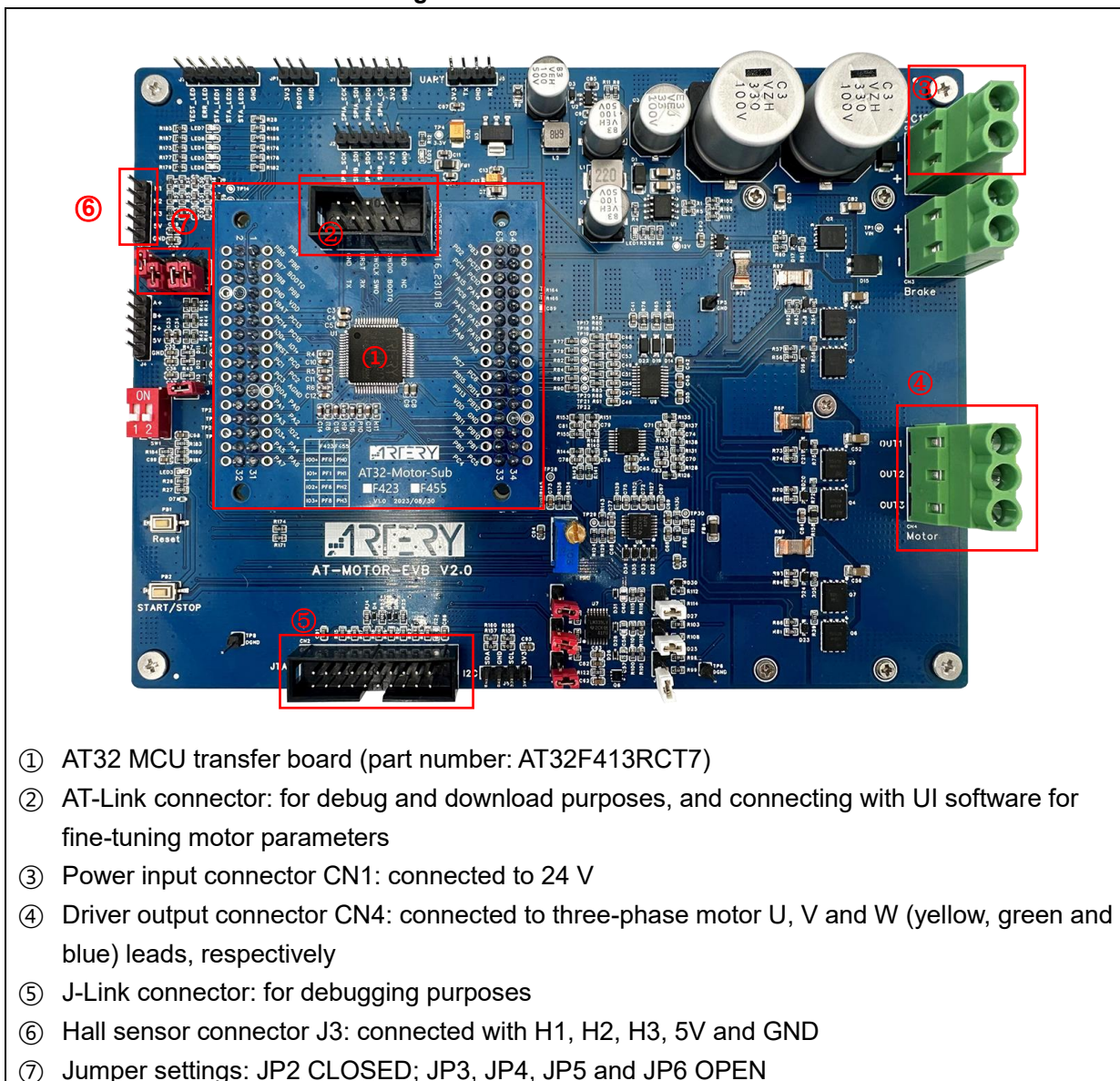


Figure 2. AT-MOTOR-EVB V2.x



2.2 Software requirements

- 1) Taking AT32F413 as an example, open AT32F413_MC_Library_Project\motor_evb_2v0\at32f413\pmsm_foc_hall_sensor_E-Bike-Scooter(note [1])
- 2) Run “ArteryMotorMonitor.exe” (installation not required)
- 3) In Keil, go to **Options - Read/Only Memory Areas** to set ROM according to the Flash memory size of each AT32 MCU (see Table 1). For example: for AT32F413RCT7 with a 256KB Flash memory, its IROM1 start address is at 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is at 0x803F800 with a size of 0x800, as shown in Figure 3. In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 4).
In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 5).

Table 1. Flash memory size and ROM configuration

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [1]: This demo does not support using of Keil complier version 5 and O0 optimization level for compiling purpose simultaneously. Thus either keil compiler version 6 or O1 optimization level is used for this purpose. As AT32 BSP source code does not support V6.15 compiler, it is recommended to use keil compiler version 5 and O1 optimization level.

Figure 3. AT32F413RCT7 ROM configuration (Keil)

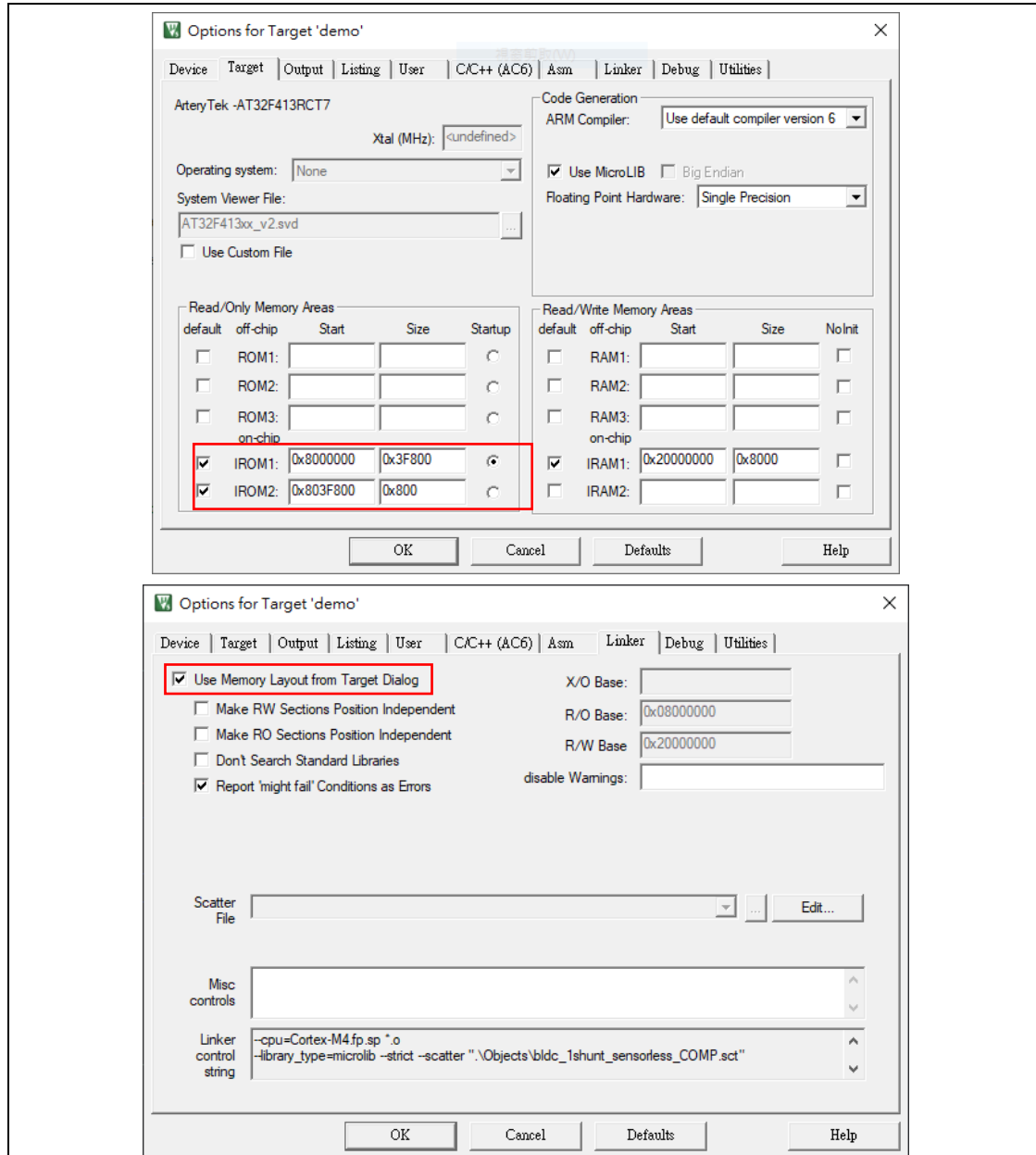


Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)

```

AT32F413xC_FLASH.ld X
25 _estack = 0x20008000; /* end of RAM */
26
27 /* Generate a link error if heap and stack don't fit into RAM */
28 _Min_Heap_Size = 0x200; /* required amount of heap */
29 _Min_Stack_Size = 0x400; /* required amount of stack */
30
31 /* Specify the memory areas */
32 MEMORY
33 {
34     FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
35     MC_DATA (r)      : ORIGIN = 0x0803F800, LENGTH = 0x800
36     RAM (xrw)        : ORIGIN = 0x20000000, LENGTH = 32K
37 }
38
39 /* Define output sections */
40 SECTIONS
41 {
42     /* The startup code goes first into FLASH */
43     .isr_vector :
44     {
45         . = ALIGN(4);
46         KEEP(*(.isr_vector)) /* Startup code */
47         . = ALIGN(4);
48     } >FLASH
49
50     .mc_data :
51     {
52         . = ALIGN(4);
53         . = ORIGIN(MC_DATA);
54         _MC_VectStoreAddr1 = .;
55         *(.MC_VectStoreAddr1);
56         . = ALIGN(4);
57         . = ORIGIN(MC_DATA) + 800;
58         _MC_VectStoreAddr2 = .;
59         *(.MC_VectStoreAddr2);
60         . = ALIGN(4);
61     } >MC_DATA
62
63     /* The program code and other data goes into FLASH */
64     .text :
65     {
66         . = ALIGN(4);
67         *(.text)           /* .text sections (code) */
68         *(.text*)          /* .text* sections (code) */
69         *(.glue_7)          /* glue arm to thumb code */
70         *(.glue_7t)         /* glue thumb to arm code */
71         *(.eh_frame)
72

```

Figure 5. AT32F413RCT7 ROM configuration (IAR)

```

1  /****ICF**** Section handled by ICF editor, don't touch! *****/
2  /*-Editor annotation file-*/
3  /* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_vl_0.xml" */
4  /*-Specials-*/
5  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
6  /*-Memory Regions-*/
7  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
8  define symbol __ICFEDIT_region_ROM_end__ = 0x0803F800 - 1;
9  define symbol __ICFEDIT_region_ROM1_start__ = 0x0803F800;
10 define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 800 - 1);
11 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
12 define symbol __ICFEDIT_region_ROM2_end__ = 0x0803F800 + 0x800 - 1;
13 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
14 define symbol __ICFEDIT_region_RAM_end__ = 0x20007FFF;
15 /*-Sizes-*/
16 define symbol __ICFEDIT_size_cstack__ = 0x400;
17 define symbol __ICFEDIT_size_heap__ = 0x200;
18 /**** End of ICF editor section. ****ICF*****/
19
20 define memory mem with size = 4G;
21 define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
22 define region ROM1_region = mem:[from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
23 define region ROM2_region = mem:[from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
24 define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
25
26 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
27 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };
28
29 initialize by copy { readwrite };
30 do not initialize { section .noinit };
31
32 place at address mem:__ICFEDIT_intvec_start__ { readonly section .intvec };
33
34
35 place in ROM_region { readonly };
36 place in ROM1_region { readonly section .MC_VectStoreAddr1 };
37 place in ROM2_region { readonly section .MC_VectStoreAddr2 };
38 place in RAM_region { readwrite,
39 block CSTACK, block HEAP };

```

3 Quick start guide

Follow the procedures below to fine-tune a PMSM motor with hall sensor by using FOC control method.

Step 1: Modify motor control drive mode and parameters according to motor parameters and application requirements (see Section 3.1).

Step 2: Connect with UI software (see Section 3.2).

Step 3: Select open loop control mode to check whether motor is running normally (see Section 3.3).

Step 4: Enable hall phase sequence self-learning (see section 3.4).

Step 5: Verify if the motor can operate based on the Hall sensor position using voltage control (see Section 3.5).

Step 6: Current control

- Motor winding parameter identification (see Section 3.6)
- Current PI controller parameter auto identification (see Section 3.6)
- ID tune: fine-tuning Kp and Ki values of current loop (see Section 3.7)
- IQ tune: fine-tuning Kp and Ki values of current loop (see Section 3.8)
- Current loop control (support speed limitation feature) (see Section 3.9)

Note: The current loop Kp and Ki values in E-Bike-Scooter project are adjusted automatically according to the bus voltage. In this case, it is impossible to perform real-time tuning of Kp and Ki values in current control mode. Therefore, it is recommended to adjust Kp and Ki values during ID tuning and IQ tuning mode before writing into the Flash.

Step 7: Tune speed control parameters, which is supported in E-Bike-Scooter mode. The speed PID controller parameters affect the response of current loop (see Section 3.10).

Step 8: Adjust field weakening control parameters as needed (see Section 3.12)

Step 9: (If the motor type is a Surface-mounted Permanent Magnet Synchronous Motor (SPMSM), skip this step.) If the motor type is an Interior Permanent Magnet Synchronous Motor (IPMSM) and requires Maximum Torque per Ampere (MTPA) control and Maximum Torque per Voltage (MTPV) control, it is necessary to perform MTPA/MTPV table creation and look-up table control (as referenced in Section 3.13).

Step 10: Tune reverse mode parameters. In reverse mode, users can set the maximum reversing speed and the reversing maximum current command (see Section 3.14).

Step 11: Tune brake regenerative mode parameters. In brake regenerative mode, users can set the braking current command (see Section 3.15).

Step 12: Brake switch operating instructions (refer to Section 3.16)

Step 13: By default, control commands are provided via UI software. To switch to external potentiometer control, adjust the control command source (see Section 3.17).

3.1 Modify motor control mode and parameters

- The *motor_control_drive_param.h* file allows users to configure the motor drive mode, motor parameters, board parameters and controller parameters.
- Users can configure the desired drive mode according to their hardware and motor configurations, as detailed below.

1. Select to use an internal crystal.

Table 2. Macro definition setting (internal crystal)

Definition	Description
INTERNAL_CLOCK_SOURCE	Use an internal crystal (disabled by default)

2. Select a motor development board (V1 or V2, optional).

Table 3. Macro definition setting (EVB version)

Definition	Description
AT_MOTOR_EVB_V1	AT-MOTOR-EVB V1.x (include mc_hwio_v1.h file)
AT_MOTOR_EVB_V2	AT-MOTOR-EVB V2.0 (include mc_hwio_v1.h file)

3. Enable or disable lower-side inverted output feature depending on the gate driver used in the user hardware circuit. For example, the U3315 in the AT32_LV_MOTOR_CONTROL_EVB is with this inverted output function, so it is pre-defined to Enable.

Table 4. Macro definition setting (lower-side MOS inverted output)

Definition	Description
GATE_DRIVER_LOW_SIDE_INVERT	Enable lower-side MOS inverted output (enabled by default; comment to disable)

4. This example is a FOC vector control demonstration project, so there is no need to make changes here. This option, however, is reserved for use in code judgment.

Table 5. Macro definition setting (vector control mode)

Definition	Description
FOC_CONTROL	Vector control mode

5. Select a current sampling method from below according to user hardware circuit and application requirements.

Table 6. Macro definition setting (current sampling method)

Definition	Description
THREE_SHUNT	3-shunt current sampling or MOS resistance current sampling (hardware circuit needs to be modified)
TWO_SHUNT	2-shunt current sampling
ONE_SHUNT	1-shunt current sampling

6. If there is a need to use 2-shunt current sampling, select one from below according to

user hardware circuit and application requirements, if not, this step is not needed.

Table 7. Macro definition setting (two-shunt current sampling method)

Definition	Description
U_V_SHUNT	U & V shunt
V_W_SHUNT	V & W shunt
U_W_SHUNT	U & W shunt

7. Based on whether the user's hardware circuit uses the low-side MOSFET's $R_{DS(on)}$ for current sampling.

Table 8. Macro definition setting (MOSFET's $R_{DS(on)}$)

Definition	Description
MOS_RDS_SHUNT	Current Sampling Utilizing Low-Side MOSFET's $R_{DS(on)}$ (disabled by default)

8. Based on whether the user's requirements call for using dual ADCs for current sampling (Applicable to MCUs with dual or more ADCs; not suitable for ONE_SHUNT configurations).

Table 9. Macro definition setting (dual ADCs current sampling)

Definition	Description
TWO_ADC_CONVERTERS	Current Sampling with Dual-ADC Synchronization (disabled by default)

9. This example is for hall sensor project, so the hall sensor is selected by default, and there is no need to make changes here. This option, however, is reserved for use in code judgment.

Table 10. Macro definition setting (hall sensor)

Definition	Description
HALL_SENSORS	Hall sensor

10. Enable D/Q-axis current signal low-pass filtering mode according to actual needs

Table 11. Macro definition setting (D/Q-axis current low-pass filter)

Definition	Description
CURRENT_LP_FILTER	D/Q-axis current low-pass filter signals (disabled by default for 3 or 2-shunt current sampling, but enabled by default for 1-shunt current sampling)

11. Enable a/b/c phase current signal low-pass filter according to user needs.

Table 12. Macro definition setting (a/b/c phase current low-pass filter)

Definition	Description
AC_CURRENT_LP_FILTER	a/b/c phase current low-pass filter (disabled by default)

12. This example is for E-Bike-Scooter project, so there is no need to make changes here. This option, however, is reserved for use in code judgment.

Table 13. Macro definition setting (E-Bike-Scooter project)

Definition	Description
E_BIKE_SCOOTER	E-bike/scooter mode

13. E-Bike-Scooter example case demonstrates how to limit the maximum current output using DC current limit feature. This feature is OFF by default, but it is enabled by setting the DC_MAX_CURRENT =1 in the motor_control_drive_param.h, see note [2].

Table 14. Macro definition setting (bus current limit)

Definition	Description
DC_CURRENT_LIMIT	DC current limit (for E_BIKE_SCOOTER only)

14. E-Bike-Scooter example case demonstrates how to use MOS's RDS(on) auto calibration feature (hardware circuit must be MOS RDS current sampling mode). This feature is OFF by default, but it is enabled by setting the RDS_AUTO_CALIBRATION =1, see note [2].

Table 15. Macro definition setting (MOS RDS(on) auto calibration)

Definition	Description
RDS_AUTO_CALIBRATION	MOS's RDS(on) auto calibration (for E_BIKE_SCOOTER only) (note [2])

Note [2]: For AT-MOTOR-EVB evaluation board, the one-shunt current sampling is not available after DC_CURRENT_LIMIT or RDS_AUTO_CALIBRATION is enabled, and the following requirements must be met: AT-MOTOR-EVB V1.x: R148=10kΩ and C91=0.1μF; AT-MOTOR-EVB V2.0: R154=10kΩ; AT-MOTOR-EVB V2.1: R140=10kΩ. Additionally, if the RDS_AUTO_CALIBRATION is to be used, the current sampling circuit must be the three-phase MOS RDS (on) current sampling mode.

15. Based on the user's motor type, select either a Surface-Mounted Permanent Magnet Synchronous Motor (SPMSM) or an Interior Permanent Magnet Synchronous Motor (IPMSM) (choose one to activate).

Table 16. Macro definition setting (motor type)

Definition	Description
SPMSM	Surface-Mounted Permanent Magnet Synchronous Motor
IPMSM	Interior Permanent Magnet Synchronous Motor

16. If the user's motor type is an Interior Permanent Magnet Synchronous Motor (IPMSM), enable the macro definition IPM_MTPA_MTPV_TABLE to generate MTPA/MTPV lookup tables

Table 17. Macro definition setting (MTPA/MTPV lookup generation)

Definition	Description
IPM_MTPA_MTPV_TABLE	MTPA/MTPV lookup generation (disabled by default)

17. If the user's motor type is an Interior Permanent Magnet Synchronous Motor (IPMSM), generate MTPA/MTPV lookup tables and store them in the Id_MTPA_LUT and Iq_MTPV_LUT matrices within the MTPA_MTPV_table.c file. Enable the macro definition IPM_MTPA_MTPV_CTRL to execute lookup table-based torque control.

Table 18. Macro definition setting (MTPA/MTPV lookup table-based torque control)

Definition	Description
IPM_MTPA_MTPV_CTRL	MTPA/MTPV lookup table-based torque control (enabled by default)

18. If the user's motor type is a Surface Permanent Magnet Synchronous Motor (SPMSM), the flux-weakening function can be optionally enabled during speed control based on user requirements.

Table 19. Macro definition setting (field weakening control)

Definition	Description
FIELD_WEAKENING	Field weakening control (disabled by default)

19. Enable current decouple feature during torque (current) control according to user needs.

Table 20. Macro definition setting (current decouple control)

Definition	Description
CURRENT_DECOUPLE_CTRL	Current decouple control (N/A for AT32L021 series) (disabled by default)

20. 17. Enable motor winding parameter identification feature. Once enabled, it is used to automatically detect RS and LS of the motor windings, and calculate current loop PI controller parameters (see Section 3.6).

Table 21. Macro definition setting (motor winding parameter identification)

Definition	Description
MOTOR_PARAM_IDENTIFY	Motor winding parameter identification (enabled by default; comment to disable)

21. 18. Enable or disable the Artery motor UI too according to user needs.

Table 22. Macro definition setting (Artery motor UI tool)

Definition	Description
USE_MOTOR_MONITOR	Enable Artery motor UI too (enabled by default; comment to disable)

Table 23 gives the motor parameters used in the demo, such as number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 23. Motor parameter definition

Definition name	Description
RS_LL	Motor line-to-line winding resistance (unit: Ω)
LS_LL	Motor line-to-line winding inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
KE	Motor KE value (unit: V/rpm) (fill in this value when current decouple control is enabled)
LD_LQ_RETIO	Ld to Lq ratio
NOMINAL_CURRENT	Nominal current (unit: ampere)
HALL_LEARN_DIR	Motor direction after hall self-learning procedure
HALL_LEARN_0_STATE	Hall state at 0 degree after Hall self-learning session
HALL_LEARN_1_STATE	Hall state at 60 degree after Hall self-learning session
HALL_LEARN_2_STATE	Hall state at 120 degree after Hall self-learning session
HALL_LEARN_3_STATE	Hall state at 180 degree after Hall self-learning session
HALL_LEARN_4_STATE	Hall state at 240 degree after Hall self-learning session
HALL_LEARN_5_STATE	Hall state at 300 degree after Hall self-learning session

3.2 Connection settings

After getting hardware and software ready, connect UI with the motor control evaluation board as follows. For details, refer to *AN0063_AT32 Motor Monitor Application Note*.

Step 1:

Connect the motor, AT-Link/J-Link and power supply to the motor control evaluation board, and connect USART interface to PC via an USB cable.

Step 2:

Use MDK to compile demo project code, and download code to the motor control evaluation board via J-Link or AT-Link.

Step 3:

Run ArteryMotorMonitor.exe.

Go to File -> Open Project, and select ArteryMotorMonitor.atmcx-> Open.

Step 4:

Click the update icon (1.) of Serial Port and select the corresponding serial port (2.); then click Open(3.) to enable real-time communication, as shown in Figure 6.

Figure 6. Connection settings

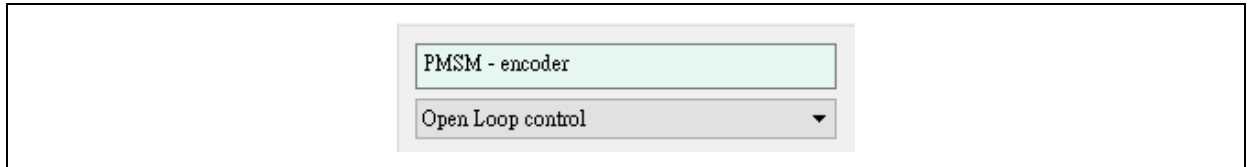


3.3 Open loop control

Using open-loop voltage control, this mode allows motor rotation without position sensors. It serves to confirm the motor's operational status and direction. The open-loop voltage and angular increment are adjusted incrementally from zero, based on the motor's speed and phase current.

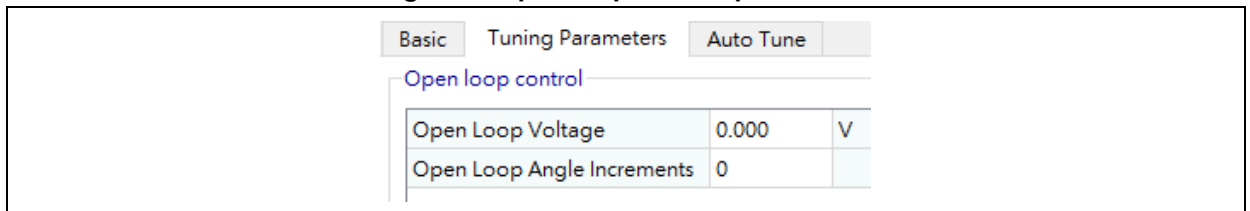
Step 1: Select "Open loop control" through the drop-down menu.

Figure 7. Select open loop control mode



Step 2: Go to "Tuning Parameters" to set "Open Loop Voltage" and "Open Loop Angle Increments". Click on "Start Motor" and monitor the current until the motor starts to run properly (set the open loop voltage value appropriately to prevent overheating damage to the motor).

Figure 8. Open loop control parameters



Tips:

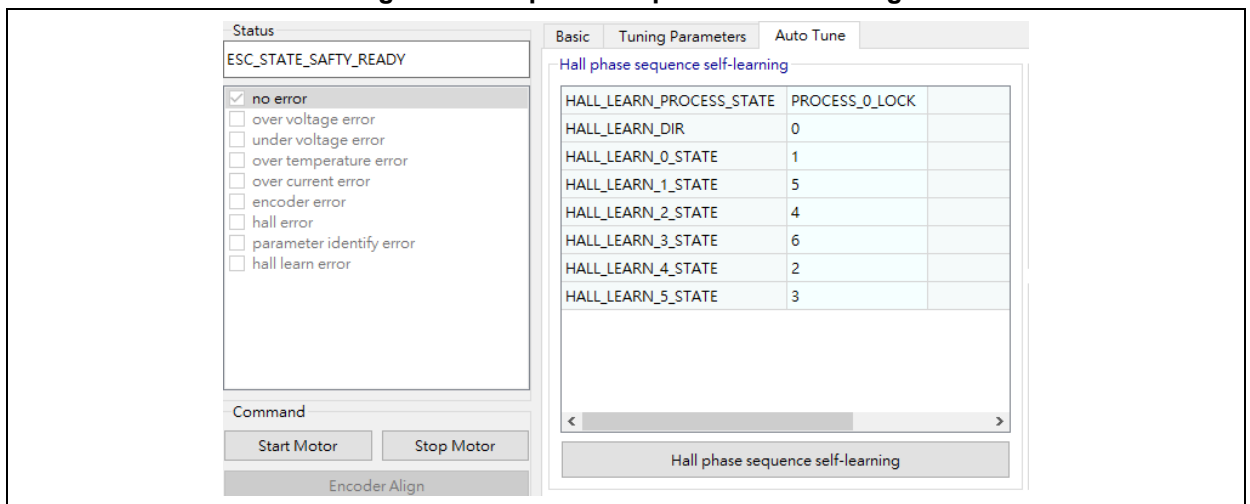
Initially, set "Open Loop Voltage" value to 0.5~2V and set "Open Loop Angle Increments" value to 1~10, and monitor whether the motor starts to run. If not, check whether the 3-phase cables are correctly connected.

3.4 Hall phase sequence self-learning

Step 1: Connect UI software.

Step 2: Enter debug mode and turn to Auto tune page. When the status is ESC_STATE_SAFETY_READY, click on "Hall phase sequence self-learning", as shown in Figure 9.

Figure 9. Hall phase sequence self-learning

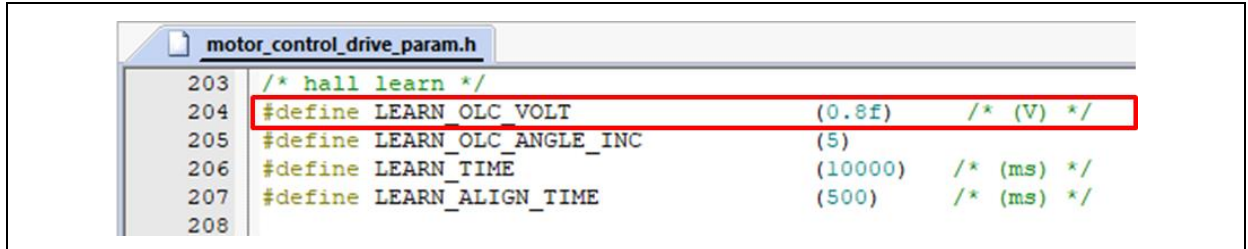


Step 3: Motor starts running in open loop mode. If the motor rotating direction is found reverse, the

user needs to wait until the motor stops before pressing “Hall phase sequence self-learning” again to adjust its rotation direction. The Hall phase sequence self-learning feature is kept active until the motor stops working.

Note: If the motor failed to spin or work normally, it is necessary to tune open loop voltage macro definition (LEARN_OLC_VOLT) and restart compiling and programming code (as shown in Figure 10), before restarting Step 2.

Figure 10. Hall phase sequence self-learning open loop voltage

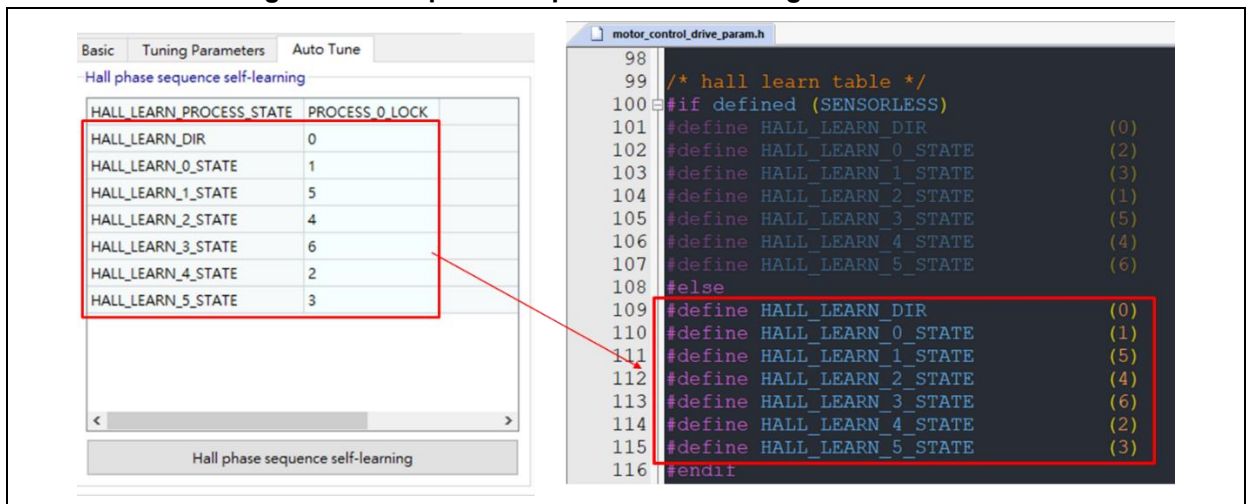


Step 4: Confirm whether the HALL_LEARN_PROCESS_STATE is “PROCESS_4_FINISH” or not.

Step 5: Write the values of the HALL_LEARN_DIR and

HALL_LEARN_0_STATE~HALL_LEARN_5_STATE to the “hall learn table” macro definition (motor_control_drive_param.h) and restart compiling and programming code, as shown in Figure 11.

Figure 11. Hall phase sequence self-learning macro definition

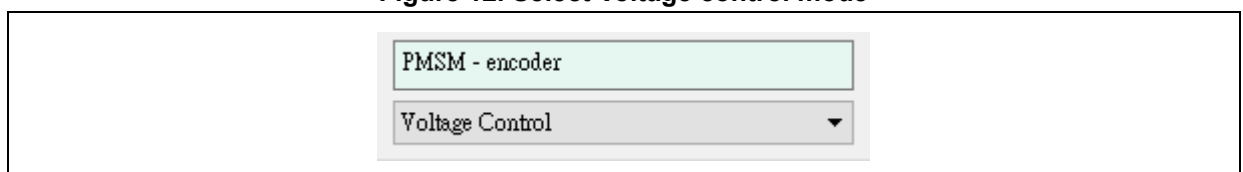


3.5 Voltage control

After Hall state self-learning operation is complete, it is possible to perform voltage control based on hall sensor position.

STEP 1. Select “Voltage Control” through the drop-down menu.

Figure 12. Select Voltage control mode



STEP 2. Use Q-axis voltage control to drive the motor, and view whether the Ia and Ib data match

the actual currents, whether the current offset value is closer to 0 and whether the Ia phase lead Ib phase by 120 degrees (see Figure 33).

Figure 13. Voltage control parameters

Voltage control		
Vq reference	1	V
Vd reference	0	V

3.6 Auto identification and tuning of parameters

Automatic parameter identification is divided into two parts, namely automatic identification of motor winding parameters and self-tuning of current PI controller parameters.

Auto identification of motor winding parameters are not required if there are RS_LL and LS_LL parameters. Users can directly modify macro definitions of RS_LL and LS_LL in *motor_control_drive_param.h* file.

Auto identification of motor winding parameters:

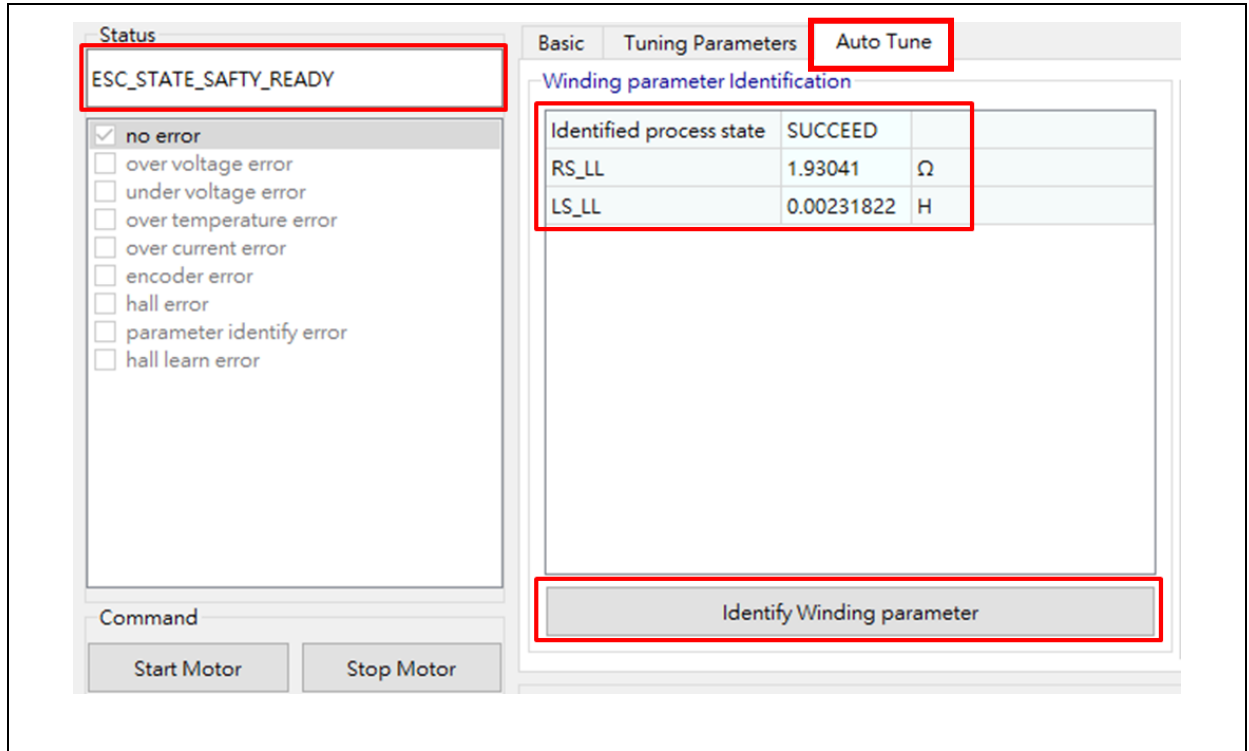
Step 1: Write the macro definition NOMINAL_CURRENT into the *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 14.

Figure 14. Nominal current macro definition

```
motor_control_drive_param.h
79  /***** Motor-related parameter *****/
80  #define POLE_PAIRS          (8/2)
81  #define RS_LL                (1.89f) /* Stator resistance(line-to-l
82  #define LS_LL                (0.002387f) /* Stator inductance(line-to-l
83
84  #define NOMINAL_CURRENT      (1.7f)
```

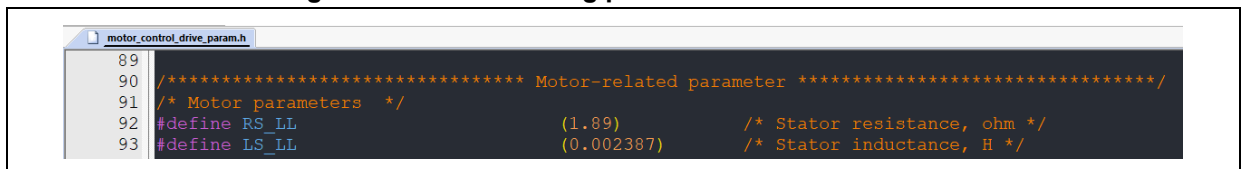
Step 2: Go to UI --> Auto Tune, and click on “Identify Winding parameter” in “safety ready” status to perform winding parameter identification, as shown in Figure 15.

Figure 15. Identify winding parameters



Step 3: Check and confirm “SUCCEED” of “Identified process state”; get the RS_LL and LS_LL values and fill them into the corresponding macro definitions in *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 16.

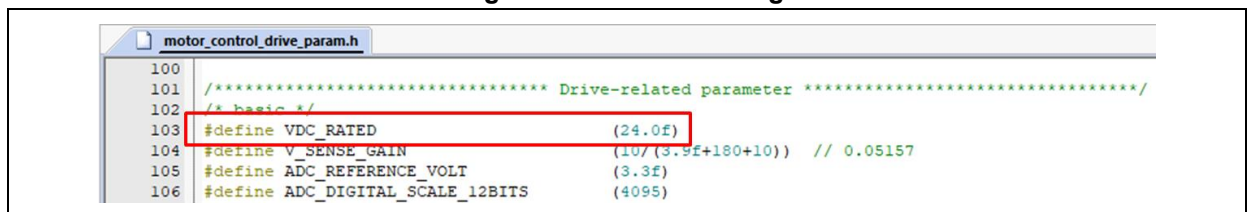
Figure 16. Motor winding parameters macro definition



Current PI controller parameter auto tuning:

Step 1: Fill in the macro definition of input DC voltage (VDC_RATED) into the *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 17.

Figure 17. Nominal voltage



Step 2: Go to “Auto Tune” page, click on “Auto-tune Current PI parameter” in “safety ready” status to perform auto tuning of current PI controller parameters. After auto tuning is completed, Q-axis and D-axis current PI controller parameters are automatically updated, as shown in Figure 18.

Figure 18. Auto tune current PI parameters

Auto-tune PI parameter of Current controller	
Torque KP	7717
Torque KI	6110
Torque KP DIV	512
Torque KI DIV	8192

Auto-tune Current PI parameter

Step 3: Write these values obtained in Figure 18 into the corresponding macro definition (motor_control_drive_param.h), then recompile and re-program code, as shown in Figure 19.

Figure 19. Macro definition of current PI parameters

```

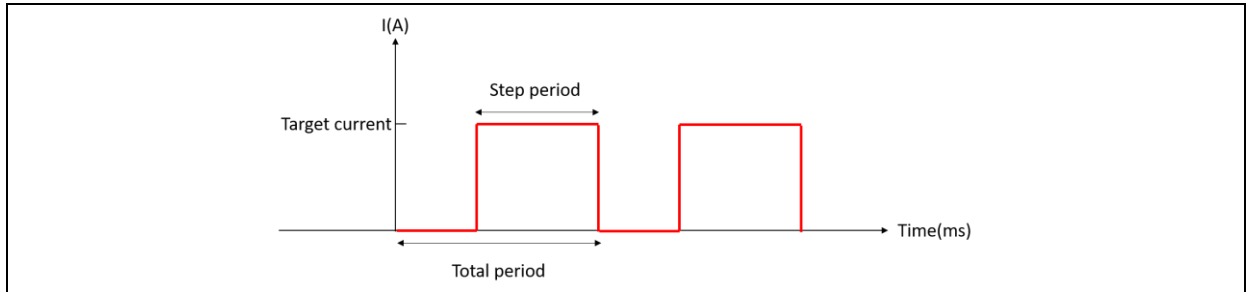
motor_control_drive_param.h
244  /* Id/Iq pid parameter */
245  #define PID_ID_KP_DEFAULT          (7717)
246  #define PID_ID_KI_DEFAULT          (6110)
247  #define PID_ID_KP_GAIN_DIV         (512)
248  #define PID_ID_KP_GAIN_DIV_LOG     (LOG2 (PID_ID_KP_GAIN_DIV) )
249  #define PID_ID_KI_GAIN_DIV         (8192)
250  #define PID_ID_KI_GAIN_DIV_LOG     (LOG2 (PID_ID_KI_GAIN_DIV) )
251
252  #define PID_IQ_KP_DEFAULT          (7717)
253  #define PID_IQ_KI_DEFAULT          (6110)
254  #define PID_IQ_KP_GAIN_DIV         (512)
255  #define PID_IQ_KP_GAIN_DIV_LOG     (LOG2 (PID_IQ_KP_GAIN_DIV) )
256  #define PID_IQ_KI_GAIN_DIV         (8192)
257  #define PID_IQ_KI_GAIN_DIV_LOG     (LOG2 (PID_IQ_KI_GAIN_DIV) )

```

3.7 ID tune

A step current is generated in IQ Tune mode, as shown in Figure 20. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of D axis current.

Figure 20. Step current diagram



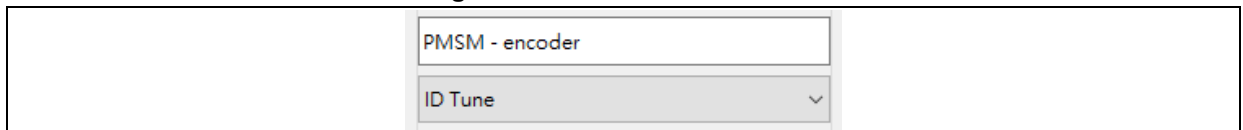
Note: It is recommended to tune D-axis current and then Q-axis current, and to set KP and KI to 0 for D-axis and Q-axis current to avoid unmatched default values affecting current parameter adjustment.

Note: It is recommended to set D-axis current command to 5-10% of the rated current and then slowly increase the D-axis current to the target value to align the rotor with D-axis, so that to protect IPM or MOSFET components from being burnt out by current surges.

Follow the steps bellow:

Step 1: Select "ID tune" through the drop-down menu.

Figure 21. Select ID Tune mode



Step 2: Go to "Tuning Parameters" window to set PID parameters and unit step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 22.

Figure 22. PID and step current parameters

Basic Tuning Parameters Auto Tune

Unit step config

Current Tune target current	0.999	A
Current Tune total period	100	ms
Current Tune step period	2	ms

ID Current control

Flux KP	7500	
Flux KI	6000	
Flux KP DIV	512	
Flux KI DIV	8192	

Step 3: Click on "Start Motor".

Step 4: Set "Flux reference(Id)" and "Flux measured(Id)" in "Diagram parameter setting", and click "Save".

Figure 23. Diagram parameter settings (ID tune)

Diagram parameter setting

Flux reference (Id)

▼

Flux measured (Id)

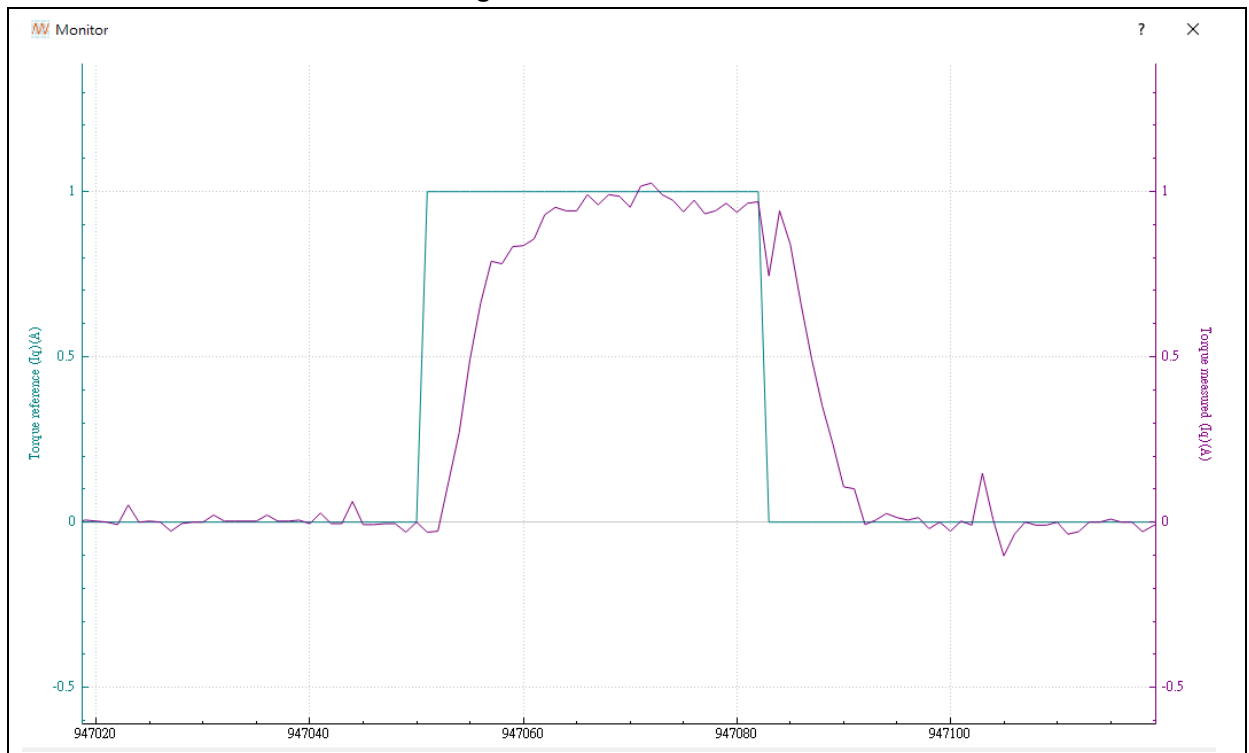
▼

Save

Step 5: Click  icon to generate waveforms.

Step 6: Check whether the current response is as expected, as shown in Figure 24. If not, click to stop the motor and repeat Step 2~Step 6.

Figure 24. ID tune waveform



Step 7: After ID tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 25.

Figure 25. D-axis current PI control parameter macro definition

```

motor_control_drive_param.h
245 #define PID_ID_KP_DEFAULT      (7500)
246 #define PID_ID_KI_DEFAULT      (6000)
247 #define PID_ID_KP_GAIN_DIV     (512)
248 #define PID_ID_KP_GAIN_DIV_LOG (LOG2(PID_ID_KP_GAIN_DIV))
249 #define PID_ID_KI_GAIN_DIV     (8192)
250 #define PID_ID_KI_GAIN_DIV_LOG (LOG2(PID_ID_KI_GAIN_DIV))
    
```

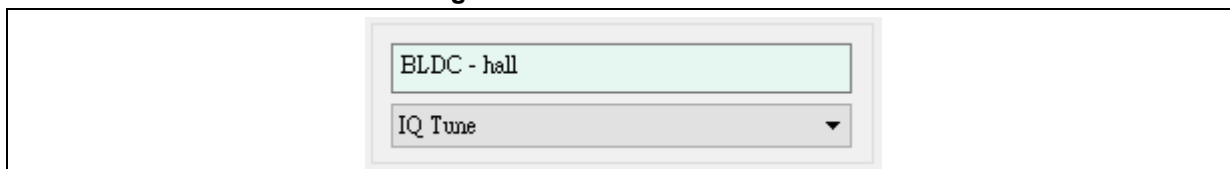
3.8 IQ tune

A step current is generated in IQ Tune mode, as shown in Figure 20. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of Q axis current.

Follow the steps bellow:

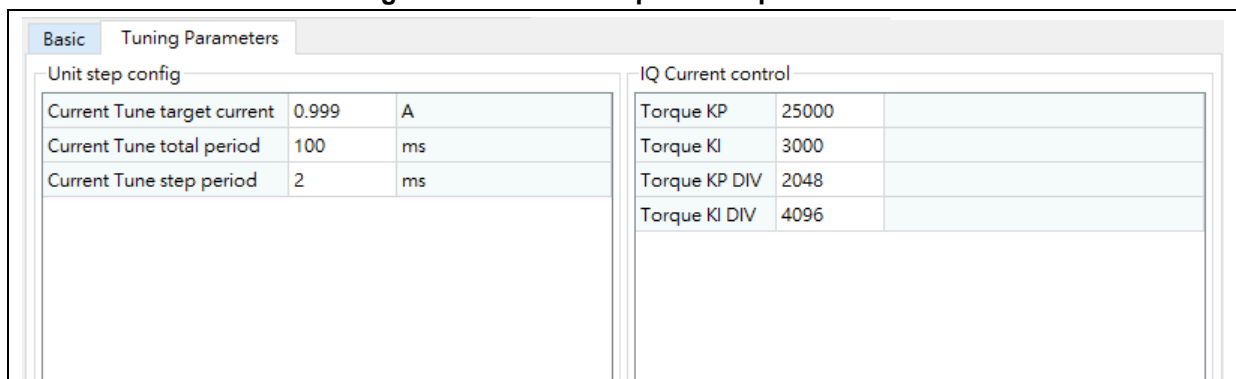
Step 1: Select "IQ tune" through the drop-down menu.

Figure 26. Select ID Tune mode



Step 2: Go to "Tuning Parameters" window to set PID parameters and unit step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 27.

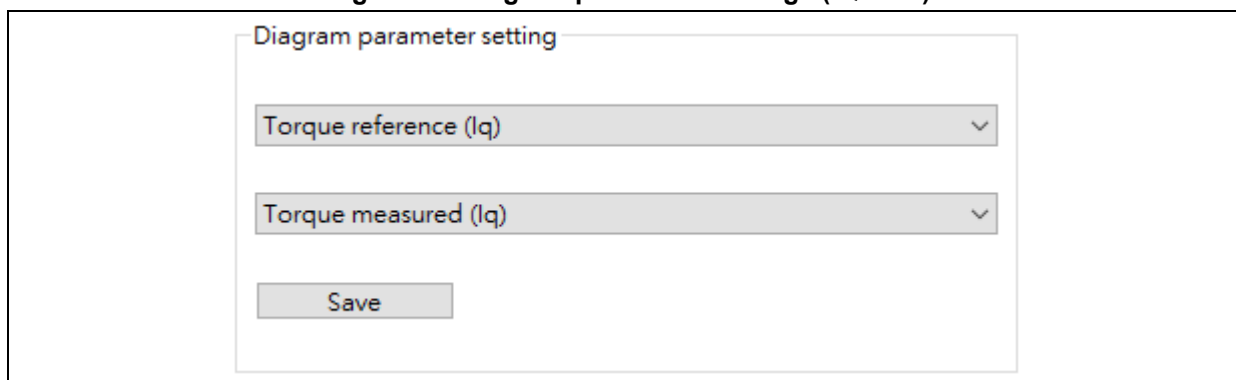
Figure 27. PID and step current parameters



Step 3: Click on "Start Motor".

Step 4: Set "Torque reference(Iq)" and "Torque measured(Iq)" in "Diagram parameter setting", and click on "Save".

Figure 28. Diagram parameter settings (IQ tune)

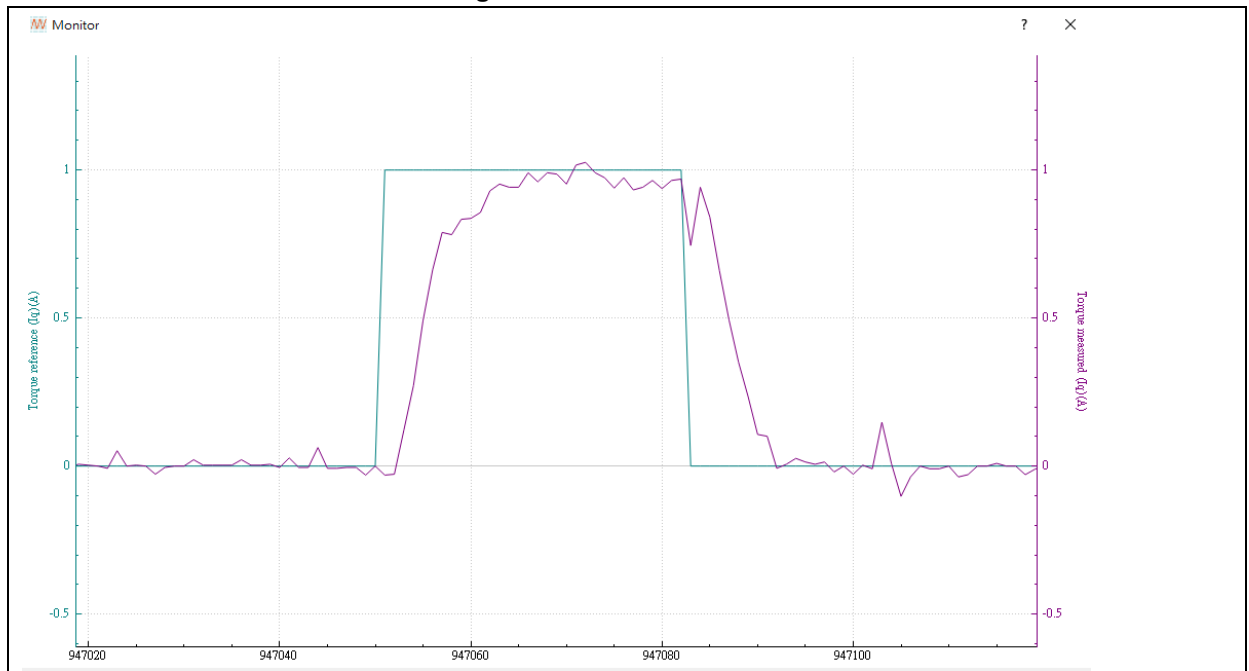


Step 5: Click  icon to generate waveforms. For more information, please refer to AN0063 AT32

Motor Monitor Application Note

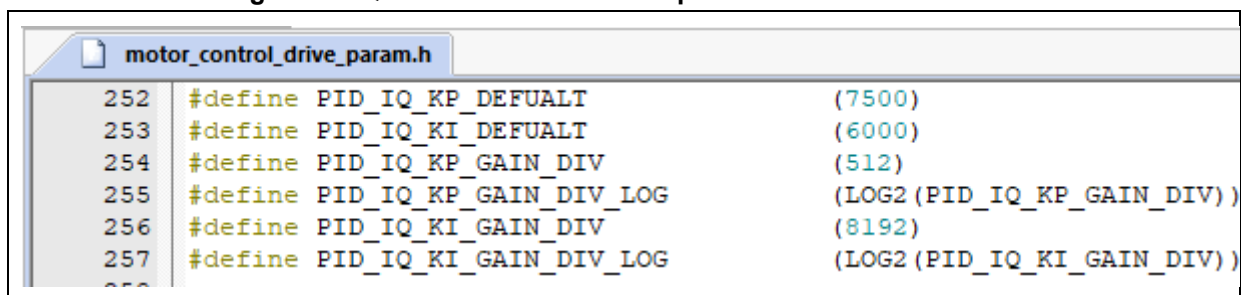
Step 6: Check whether the current response is as expected, as shown in Figure 29. If not, click to top the motor and repeat Step 2~Step 6.

Figure 29. IQ tune waveform



STEP 7: After IQ tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 30.

Figure 30. Q-axis current PI control parameter macro definition



3.9 Current loop control

In E-bike-scooter mode, the current loop control is used to control torque by tuning target current and limit the maximum speed. The results are reflected through waveforms.

Follow the steps below:

Step 1: Set MAX_SPEED_RPM(motor_control_drive_param.h), and re-start compiling and programming code

Step 2: Select "Torque Control" through the drop-down menu.

Step 3: Set software control as control source and configure target current reference (Torque reference). In doing so, the motor then starts running without pressing "Start motor" operation. It stops running when torque reference is to 0.

Figure 31. Set target current

Torque reference (Iq)	0.100	A
Flux reference (Id)	0.000	A

Step 4: Select "Ia" and "Ib", and click on "Save".

Figure 32. Diagram parameter settings (current loop debugging)

Diagram parameter setting

Ia

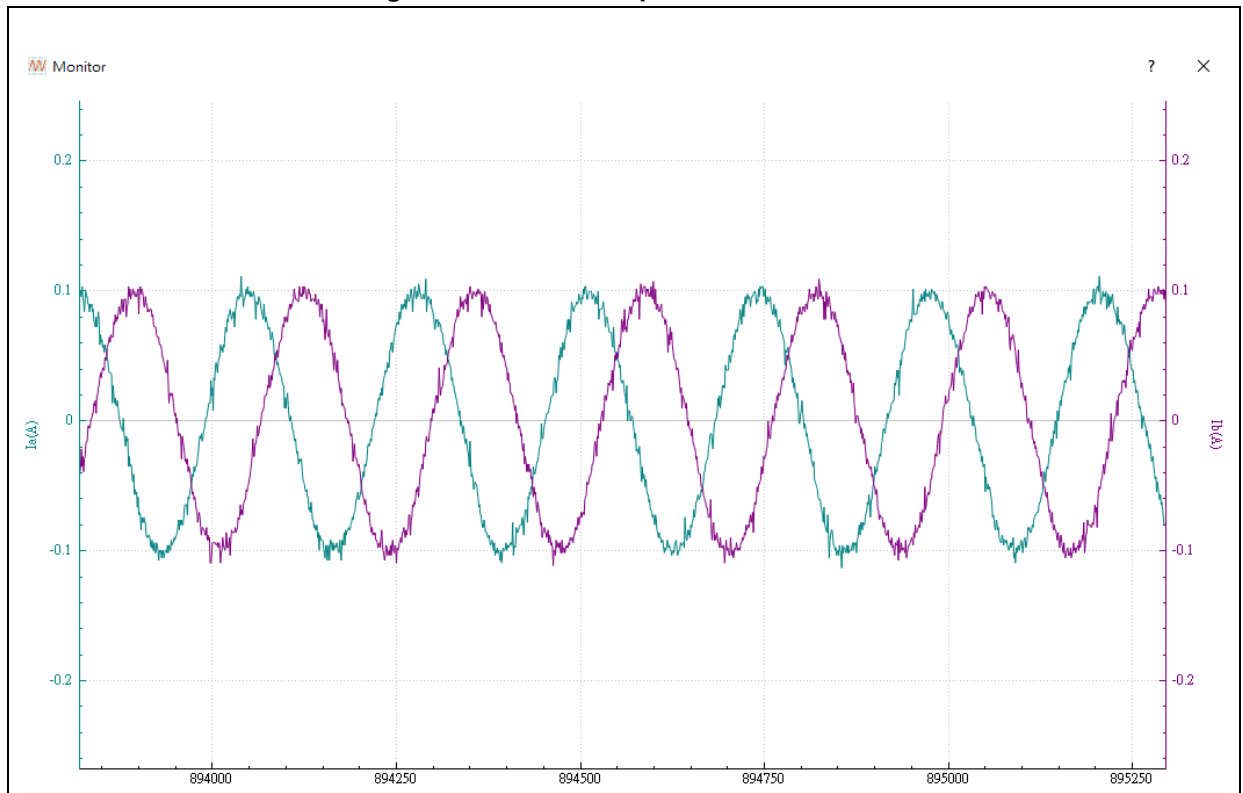
Ib

Save

Step 5: Click on “” to open a waveform window.

Step 6: Check if the current response is as expected, as shown in Figure 33.

Figure 33. Current loop control waveform



Step 7: Speed response results can also be seen in current loop mode according to step 8~ step 11.

Step 8: Set “Speed reference” and “Speed measured”, and click on “Save”.

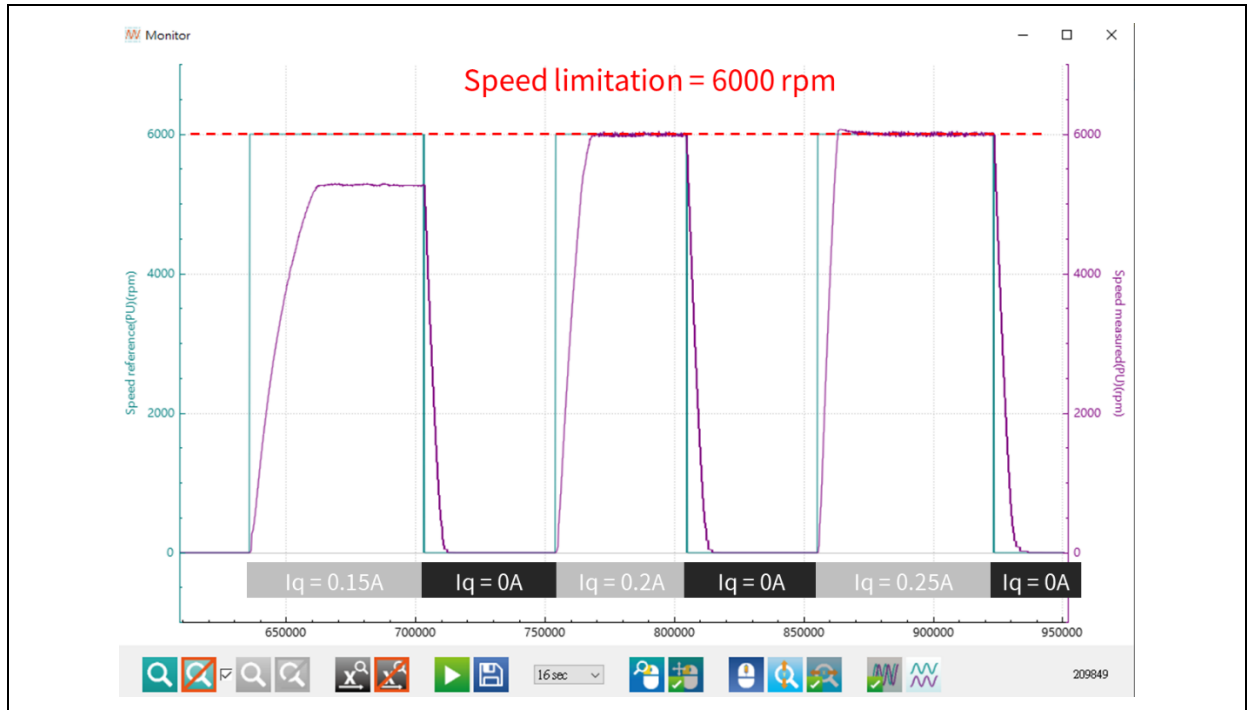
Step 9: Click on “” to open a waveform window.

Step 10: Set different torque reference values and view if the speed waveforms are as expected, as shown in Figure 34. When MAX_SPEED_RPM to 6000rpm and $I_q = 0.15A$, it is not possible to achieve 6000rpm due to too small torque; when $I_q = 0.2A$, the motor can run up to 6000rpm; when $I_q = 0.25A$, the motor runs within up to 6000rpm, with a larger rising speed slope. In most cases, the bigger the target current, the faster the speed. If the motor speed is below expectations, it is necessary to tune speed control loop parameters (see Section 3.10 for details).

Note: In this example, when the I_q current command is 0.2A or higher, the motor speed will reach and be capped at the maximum speed limit. If the speed is already at the maximum limit (indicating that $I_q < 0.2A$ is sufficient to

achieve it), the I_q command will dynamically adjust based on the output of the speed PI controller rather than being a fixed value.

Figure 34. Speed response waveforms



3.10 Speed loop control

Speed loop control is also supported in E-Bike-Scooter mode. Tuning speed PID controller parameters affects the response of current loop. In Speed control mode, the user can set speed PID parameters, acceleration and deceleration, and check the response through waveform. Follow the steps below:

Step 1: Select “Speed Control” through the drop-down menu.

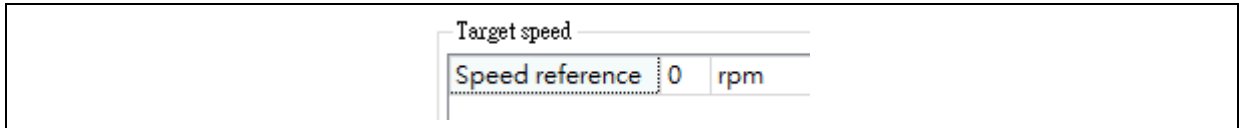
Step 2: Go to “Tuning Parameters” window, and set speed PID parameters, acceleration and deceleration.

Figure 35. PID parameters, acceleration and deceleration

Speed control		
Speed KP	1000	
Speed KI	4	
Speed KP DIV	1024	
Speed KI DIV	1024	
Speed acceleration	8	rpm/ms
Speed deceleration	8	rpm/ms

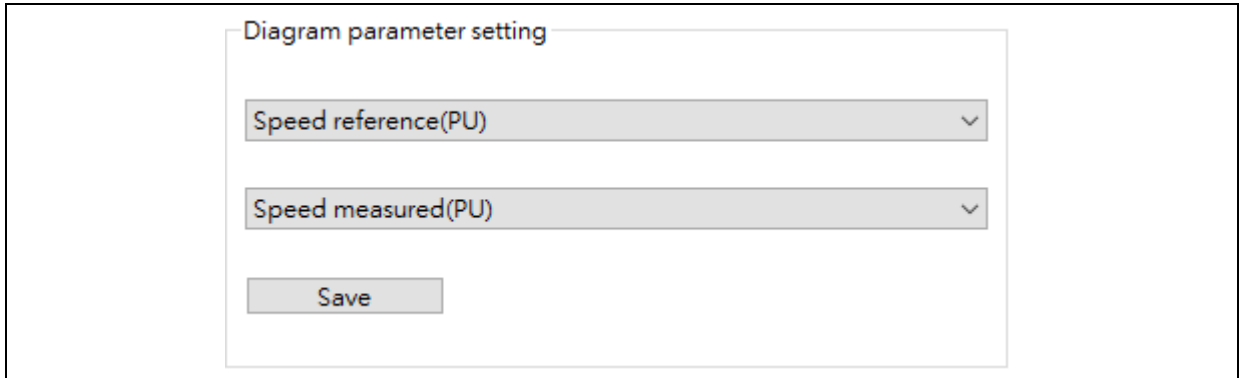
Step 3: Select “software control” in “Control source” and set the target speed (Speed reference). In doing so, the motor then starts running without pressing “Start motor” operation. It stops running when speed reference is to 0.

Figure 36. Set target speed



Step 4: Set “Speed reference” and “Speed measured” in “Diagram parameter setting”.

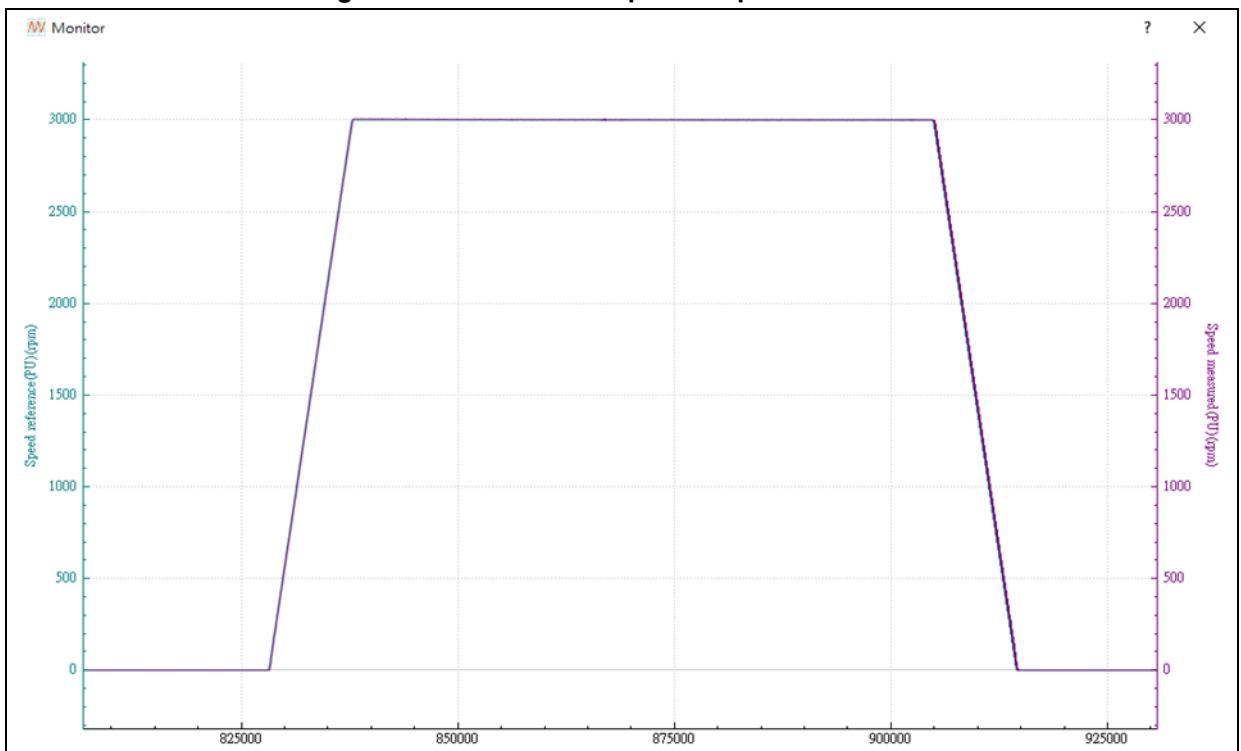
Figure 37. Diagram parameter setting (speed loop control)



Step 5. Click  icon to generate waveforms.

Step 6. Check if the speed response is as expected, as shown in Figure 38. If not, click to stop the motor and repeat STEP 2~STEP 6.

Figure 38. Waveform in speed loop control mode



Note: The maximum displayed values for speed waveforms (Speed Reference (PU) and Speed Measured (PU)) are defined as $1.25 \times \text{MAX_SPEED_RPM}$, with the minimum being $-1.25 \times \text{MAX_SPEED_RPM}$. If values exceed these limits, the waveform display will clip (overflow), though the actual numerical values remain unaffected. It is recommended to set MAX_SPEED_RPM to the motor's rated speed to avoid display anomalies.

3.11 Position loop control

Position loop control is not supported in E-Bike-Scooter mode.

3.12 Flux-Weakening Control (Surface-Mounted Permanent Magnet Synchronous Motor, SPMSM)

This mode supports Surface-Mounted Permanent Magnet Synchronous Motors (SPMSM). During speed-loop control, it injects negative d-axis current commands to weaken the permanent magnet flux linkage, thereby extending the motor's operational speed range. Adjustable parameters include the flux-weakening control (FWC) PID gains and the maximum negative d-axis current limit. After tuning, the dynamic response can be visualized via waveform plotting. Detailed implementation steps are as follows:

Step 1: Select "Speed Control" through the drop-down menu.

Step 2. Navigate to the Tuning Parameters page, configure the maximum d-axis (flux-weakening) current command and adjust the flux-weakening PID parameters.

Note: Configure the maximum d-axis (flux-weakening) current command according to the motor's rated specifications. Exceeding this limit may cause irreversible demagnetization of permanent magnets or thermal damage due to excessive copper/core losses.

Figure 39. FWC PID gains and the maximum negative d-axis current limit

-Flux weakening tuning		
Max. Flux weakening Current	1.4	A
Flux weakening KP	300	
Flux weakening KI	100	
Flux weakening KP DIV	2048	
Flux weakening KI DIV	32768	

Step 3. Set Control Source to Software Control and input the Speed Reference value. For example: 8000rpm.

Note: The flux-weakening control (FWC) is triggered only when the motor speed exceeds the base speed (i.e., the maximum sustainable speed under rated flux linkage).

Figure 40. Target speed setting (FWC)

Target speed	
Speed reference	0 rpm

Step 4: Select Speed reference(PU) and Speed measured(PU), and click on "Save".

Figure 41. Diagram parameter settings (speed control)

Diagram parameter setting

Speed reference(PU) ▼

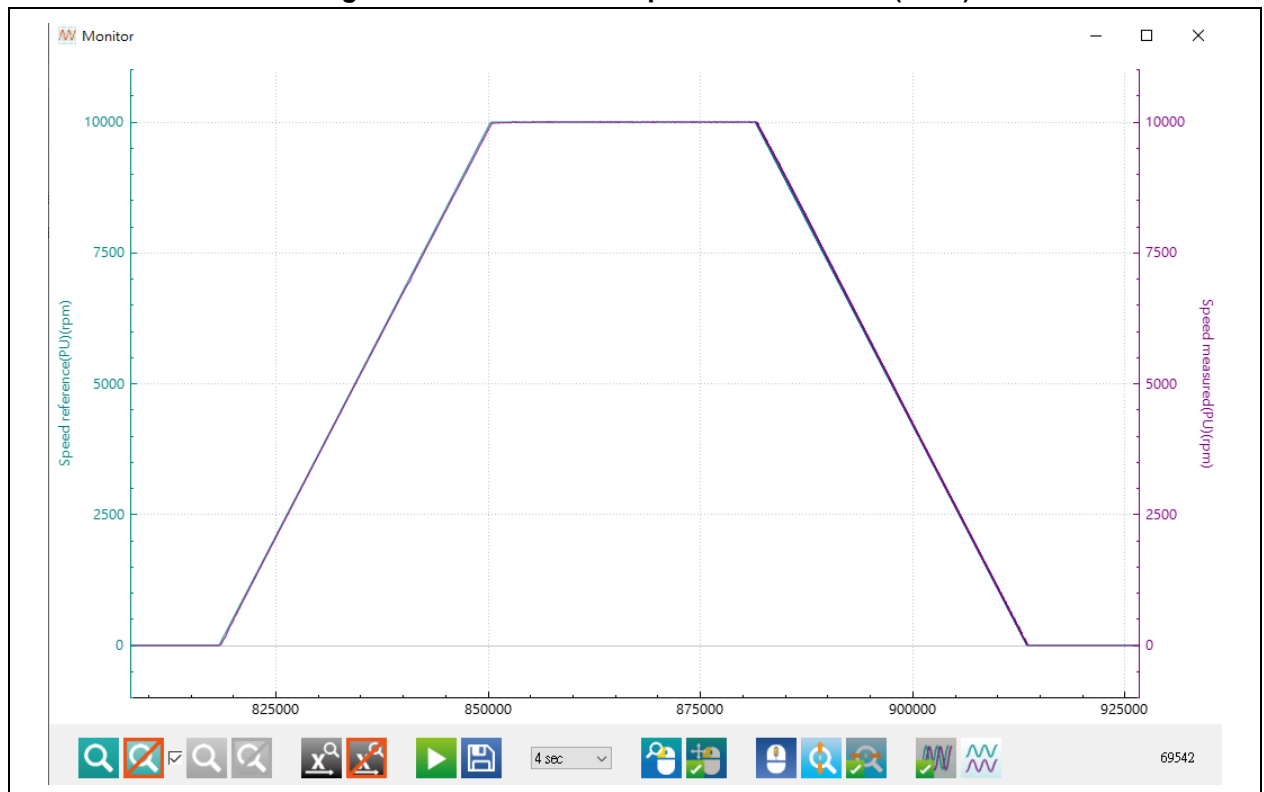
Speed measured(PU) ▼

Save

Step 5: Click  icon to generate waveforms.

Step 6: Click on “Start motor” to check whether the speed response is as expected, as shown in Figure 42. If not, click to stop the motor and repeat Step 2~Step 6.

Figure 42. Waveforms in speed control mode (FWC)



3.13 MTPA/MTPV Control (Interior Permanent Magnet Synchronous Motor, IPMSM)

MTPA (Maximum Torque Per Ampere) is an optimized control strategy for Interior Permanent Magnet Synchronous Motors (IPMSM) in the base speed region (constant torque zone). Its goal is to minimize the stator current amplitude for a given torque demand by adjusting the d-axis and q-axis current components (I_d , I_q), ensuring the motor delivers the target torque with minimal current.

MTPV (Maximum Torque Per Voltage) is a deep flux-weakening control strategy for the high-speed

region, where the motor maximizes torque output under voltage constraints by adjusting I_d and I_q . At high speeds, the back-EMF increases, reducing voltage margin. This necessitates increasing the negative I_d current to weaken the magnetic field and extend the speed range.

Why MTPA/MTPV is exclusive to IPMSM:

IPMSM relies on salient-pole characteristics ($L_d < L_q$) to generate reluctance torque, which is critical for MTPA/MTPV.

Surface PMSM (SPMSM) cannot leverage this due to structural limitations ($L_d \approx L_q$) making MTPA/MTPV ineffective.

Implementation Steps for MTPA/MTPV Table Generation:

Step 1. Define parameters for table generation: Maximum torque, Maximum speed, Torque step size and Speed step size. If a specific torque/speed value is not explicitly listed in the table, the corresponding d-axis and q-axis current components (I_d , I_q) will be calculated via interpolation.

Step 2. Generate I_q _table and I_d _table (as shown in Figure 43):

Example parameters for table generation:

Maximum torque: 5 Nm

Torque step size: 0.5 Nm

Maximum speed: 5000 rpm

Speed step size: 500 rpm

Figure 43. Example table for I_q _table and I_d _table

Torque(Nm) Speed(rpm)	0	0.5		5
500	$I_{q_{pu1}}$	$I_{q_{pu2}}$		
1000				
⋮				
5000				

Torque(Nm) Speed(rpm)	0	0.5		5
500	$-I_{d_{pu1}}$	$-I_{d_{pu2}}$		
1000				
⋮				
5000				

Note: The conversion formula between per-unit current values and actual physical current values is:

$$I(A) = \text{current_base} \cdot I_{pu} / 32767$$

Step 3. Configure Motor Type as SPMSM in motor_control_param.h, Disable FIELD_WEAKENING and IPM_MTPA_MTPV_TABLE (as shown in Figure 44)

Figure 44. Macro definitions for MTPA/MTPV table generation

```

95  /* motor type */
96  #define SPMSM
97  //#define IPMSM
98
99
100 #if defined SPMSM
101  //#define FIELD_WEAKENING
102  //#define IPM_MTPA_MTPV_TABLE
103 #elif defined IPMSM
104  #define IPM_MTPA_MTPV_CTRL
105 #endif

```

Step 4. Mount the motor on the dynamometer platform and verify mechanical alignment to minimize vibration and torque measurement errors. Complete speed-loop tuning to ensure stable motor operation under both no-load and load conditions.

Step 5. Enable IPM_MTPA_MTPV_TABLE Macro and entering the debug mode

Figure 45. Enable the macro definition IPM_MTPA_MTPV_TABLE

```

#if defined SPMSM
  //#define FIELD_WEAKENING
  #define IPM_MTPA_MTPV_TABLE
  #elif defined IPMSM
    #define IPM_MTPA_MTPV_CTRL
  #endif

```

Step 6. Configure -Id current reference (current.lqdref.d) via watch window, The Iq reference (current.lqdref.q) is automatically generated by the speed PI controller based on the speed error and tuning gains.

Figure 46. Set the per-unit value for the Id current command

Name	Value
current	0x2000005C ¤t
lbus	0x2000005C ¤t
labc	0x20000066
labc_shunt	0x2000006C
lalphabeta	0x20000072
lqd	0x20000076
lqdref	0x2000007A
q	0
d	0

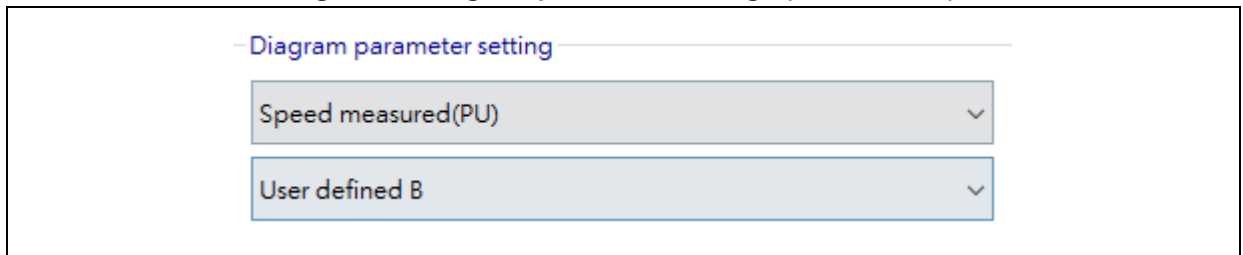
Step 7. Align the motor encoder to ensure accurate speed feedback for control stability and enter the speed closed-loop control to prepare for MTPA/MTPV table generation. (Prior to table creation, connect the motor to a dynamometer and run it at high speed for 10-20 minutes to achieve thermal stabilization)

Step 8. Follow the table in Figure 43 to identify the optimal Iq and -Id values that minimize the resultant

current I_s across different speeds and torques, starting with mid-range conditions (2500 rpm, 2.5 Nm), while monitoring whether the speed reaches the commanded value and increasing the -Id command if necessary to enhance speed performance.

Step 9. Observe the speed waveform and I_s waveform (User-defined B) on the UI to identify the minimum resultant current I_s that achieves the target speed/torque, which corresponds to the optimal I_q and -Id values.

Figure 47. Diagram parameter settings (MTPA/MTPV)



Step 10. After completing the table generation, populate the table indices into the macro definitions in the motor_control_param.h header file as shown in Figure 48. The values for MTPA_MTPV_TABLE_ROWS_NUM and MTPA_MTPV_TABLE_COLS_NUM can be determined from Figure 49.

Figure 48. MTPA/MTPV table indices

```
/* MPTA MTPV table parameters for IPMSM use only */
#define MTPA_MTPV_TABLE_MIN_SPEED      (500) /* rpm */
#define MTPA_MTPV_TABLE_MAX_SPEED      (5000) /* rpm */
#define MTPA_MTPV_TABLE_SPEED_STEP     (500) /* rpm */
#define MTPA_MTPV_TABLE_MAX_TORQUE     (5.0f) /* Nm */
#define MTPA_MTPV_TABLE_TORQUE_STEP    (0.5f) /* Nm */
#define MTPA_MTPV_TABLE_ROWS_NUM       (10) /* speed steps */
#define MTPA_MTPV_TABLE_COLS_NUM       (11) /* Torque steps */
```

Figure 49. MTPA/MTPV table row and column count

		COLS	1	2	3	4	5	6	7	8	9	10	11
	Torque→	Speed ↓	0	0.5	1	1.5	2	2.5	3	3.5	4	4.5	5
1	500												
2	1000												
3	1500												
4	2000												
5	2500												
6	3000												
7	3500												
8	4000												
9	4500												
10	5000												

Step 11. After table creation, populate the `lq_table` and `ld_table` values into the `ld_MTPA_LUT[]` and `lq_MTPA_LUT[]` matrices in the `MTPA_MTPV_table.c` file.

Step 12. In the `motor_control_param.h` header file, select the IPMSM macro and define `IPM_MTPA_MTPV_CTRL` to enable torque control mode. Inputting torque commands will trigger table-based MTPA/MTPV control.

Step 13. During torque control, verify the Torque reference waveform and the magnitudes of the `lq`/`ld` current waveforms using a UI monitoring tool to ensure they align with expectations.

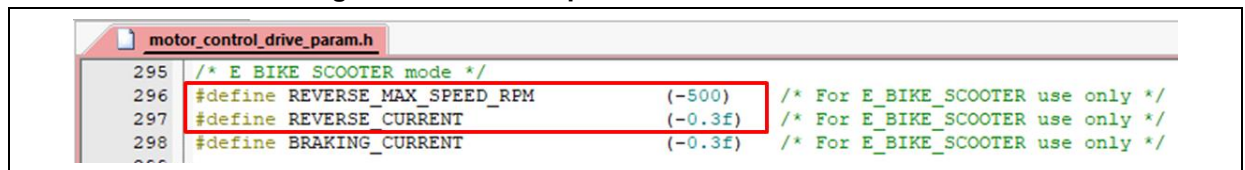
Step 14. When using `IPM_MTPA_MTPV_CTRL`, the speed loop output command is Torque reference (Nm), whereas in SPMSM control it is `lq` reference. Recalibrate the speed PI control parameters for this mode due to the fundamental difference in command logic.

3.14 Reverse mode

Reverse control mode is supported in E-Bike-Scooter mode. Follow the steps below:

Step 1: Set `REVERSE_MAX_SPEED_RPM` and `REVERSE_CURRENT`, and re-start compiling and programming code before entering debug mode, as shown in Figure 50.

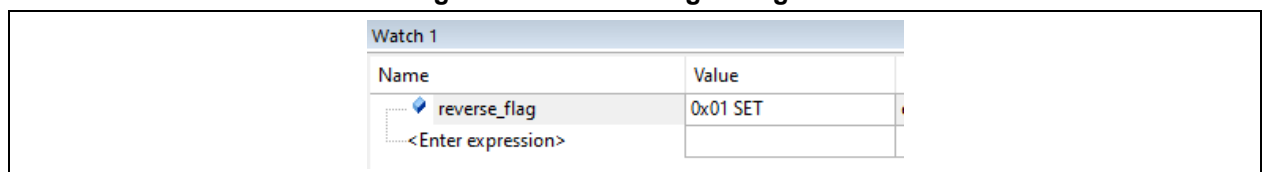
Figure 50. Reverse speed/current macro definition



Step 2: Select "Torque Control" or "Speed Control" through the drop-down menu.

Step 3: As shown in Figure 51, set `reverse_flag` = 1 to enter reverse mode. When `reverse_flag` = 0, it exists reverse mode and enters general mode (when external command control is selected, set 1=ON for SW1 to set `reverse_flag` = 1 and set 1=OFF to clear `reverse_flag` = 0).

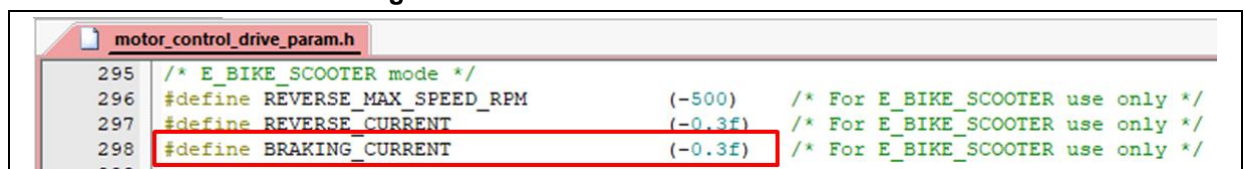
Figure 51. Reverse flag configuration



3.15 Brake recharging mode

Brake regenerative mode is supported in E-bike-scooter current loop control mode by setting `BRAKING_CURRENT` and re-starting compiling and programming code, as shown in Figure 52. When the motor runs normally at a speed higher than `STABLE_SPEED_RPM`, setting `lq`=0 in Torque Control mode will stall it and produce recharging current to the power supply source.

Figure 52. Brake current macro definition



3.16 Brake switch

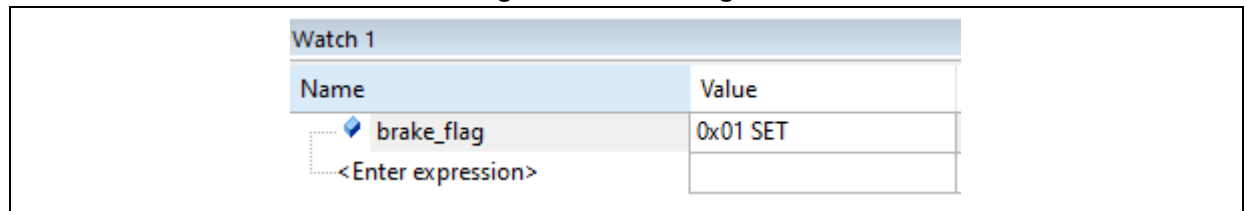
Brake switch control is supported in E-bike-scooter. Follow the steps below:

Step 1: Enter "Debug mode".

Step 2: Select "Torque Control" through the drop-down menu, select software control as the control source and set the target current reference (Torque reference), or select "Speed Control" through the drop-down menu, select software control as the control source and set the target speed reference"; meanwhile, keep the motor in running state.

Step 3: As shown in Figure 53, set brake_flag = 1 to enter brake mode. When brake_flag = 0, it exists brake mode and enters general mode (when external command control is selected, set 2=ON for SW1 to set brake_flag = 1 and set 2=OFF to clear brake_flag = 0).

Figure 53. Brake flag set



3.17 Anti-Theft mode

In E-Bike-Scooter mode, the anti-theft function could be activated in torque control mode. Once triggered, the motor will be locked, preventing the wheel from rotating or the motor from starting, making it impossible to push or ride the E-Bike normally. The parameter settings are defined by macros in motor_control_drive_param.h as follows:

MAX_LOCK_CURRENT: Maximum anti-theft current command (unit: ampere)

ANTI_THEFT_INIT_CURRENT: Initial anti-theft current value (unit: ampere)

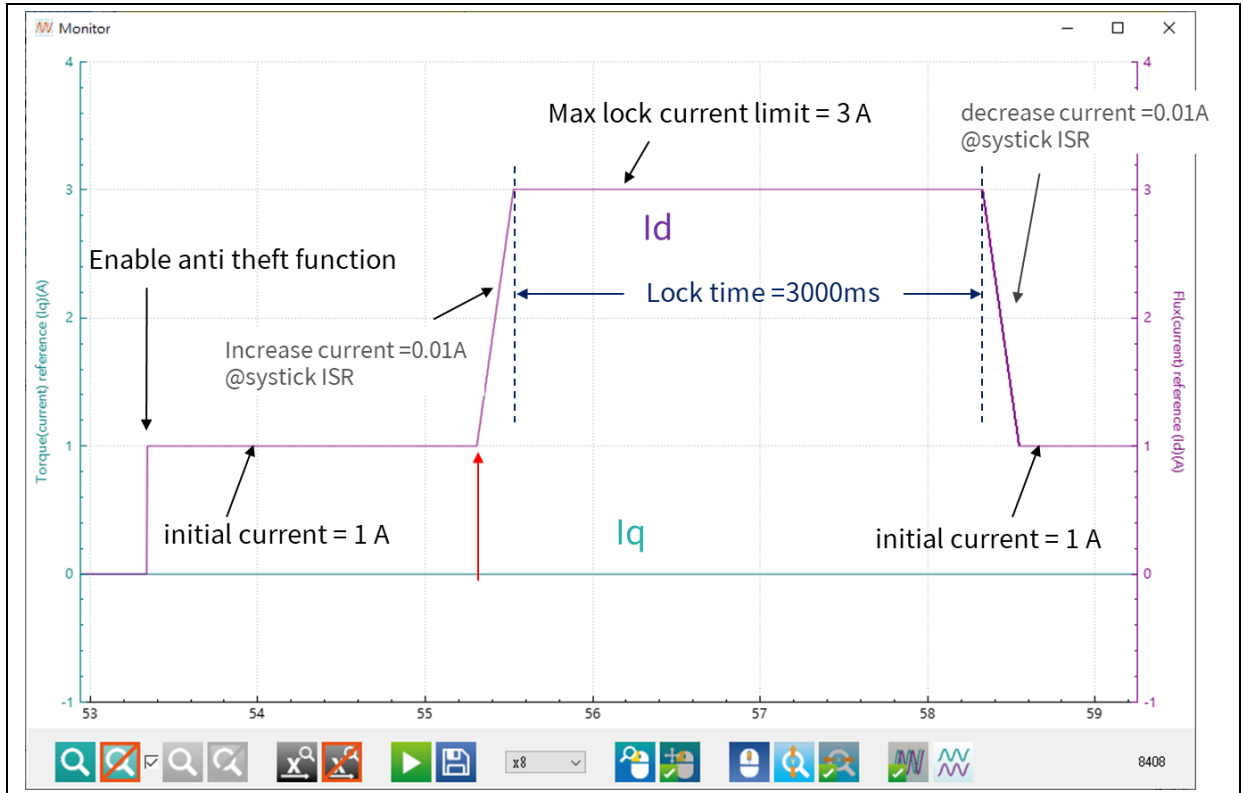
ANTI_THEFT_INC_CURRENT: Current increment per millisecond during anti-theft (unit: ampere)

ANTI_THEFT_DEC_CURRENT: Current decrement per millisecond during anti-theft (unit: ampere)

ANTI_THEFT_LOCK_TIME: Maximum current lock release time for anti-theft (unit: ms)

As shown in Figure 54, when the anti-theft function is activated, the controller injects an initial current ANTI_THEFT_INIT_CURRENT (e.g., 1A) into the motor, creating slight resistance and making the wheel difficult to rotate. If wheel movement is detected, the output current increases by ANTI_THEFT_INC_CURRENT (e.g., 10mA/ms) per millisecond, continuously enhancing the resistance. The current value rapidly approaches MAX_LOCK_CURRENT (e.g., 3A) within tens or hundreds of milliseconds until the motor is completely locked. The locked state lasts for ANTI_THEFT_LOCK_TIME (e.g., 3000ms). After the lock duration, the current gradually decreases by ANTI_THEFT_DEC_CURRENT (e.g., 10mA/ms) per millisecond until it returns to the initial current value ANTI_THEFT_INIT_CURRENT, preventing motor overheating damage due to prolonged high-current locking or excessive battery energy consumption.

Figure 54. Id current during anti-theft



Detailed operating steps:

Step 1: Enter "Debug Mode"

Step 2: Select "Torque Control" through the drop-down menu

Step 3: As shown in Figure 55, set the anti-theft flag `anti_theft_flag = 1` in the watch interface of the debug mode to activate anti-theft mode. The system returns to normal mode when `anti_theft_flag = 0`.

Figure 55. Anti-theft flag set

Watch 1	
Name	Value
☒ <code>anti_theft_flag</code>	0x01 SET
<Enter expression>	

3.18 Parking mode

Parking mode is supported under torque control mode in E-Bike-Scooter mode. When the E-Bike is stationary, the mode can be manually activated to lock the motor, preventing the E-Bike from rolling. The macro definitions for parameter settings are as follows (in `motor_control_drive_param.h`):

`MAX_LOCK_CURRENT`: Maximum parking current command (unit: ampere)

`PARKING_LOCK_INIT_CURRENT`: Initial parking current value (unit: ampere)

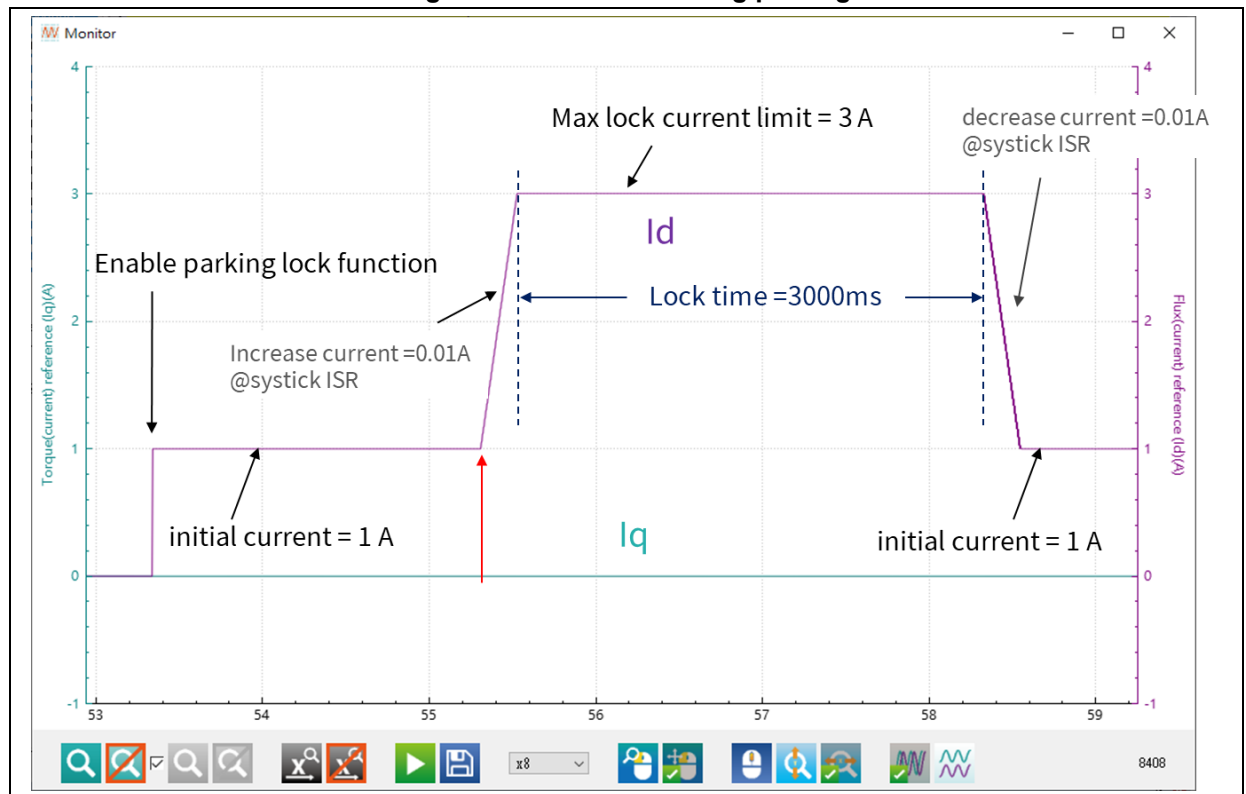
`PARKING_LOCK_INC_CURRENT`: Current increment per millisecond during parking (unit: ampere)

`PARKING_LOCK_DEC_CURRENT`: Current decrement per millisecond during parking (unit: ampere)

`PARKING_LOCK_TIME`: Maximum current lock release time for parking (unit: ms)

As shown in Figure 56, when parking mode is activated, the controller injects an initial current PARKING_LOCK_INIT_CURRENT (e.g., 1A) into the motor, generating slight resistance and making the wheel difficult to rotate. If wheel movement is detected, the output current increases by PARKING_LOCK_INC_CURRENT (e.g., 10mA/ms) per millisecond, continuously enhancing the resistance. The current rapidly approaches MAX_LOCK_CURRENT (e.g., 3A) within tens or hundreds of milliseconds until the motor is completely locked. The locked state lasts for PARKING_LOCK_TIME (e.g., 3000ms). After this duration, the current gradually decreases by PARKING_LOCK_DEC_CURRENT (e.g., 10mA/ms) per millisecond until it returns to the initial value PARKING_LOCK_INIT_CURRENT, preventing motor overheating damage or excessive battery drain due to prolonged high-current locking.

Figure 56. Id current during parking



Detailed operating steps:

Step 1: Enter "Debug Mode"

Step 2: Select "Torque Control" through the drop-down menu

Step 3: As shown in Figure 57, set the anti-theft flag anti_theft_flag = 1 in the watch interface of the debug mode to activate anti-theft mode. The system returns to normal mode when anti_theft_flag = 0.

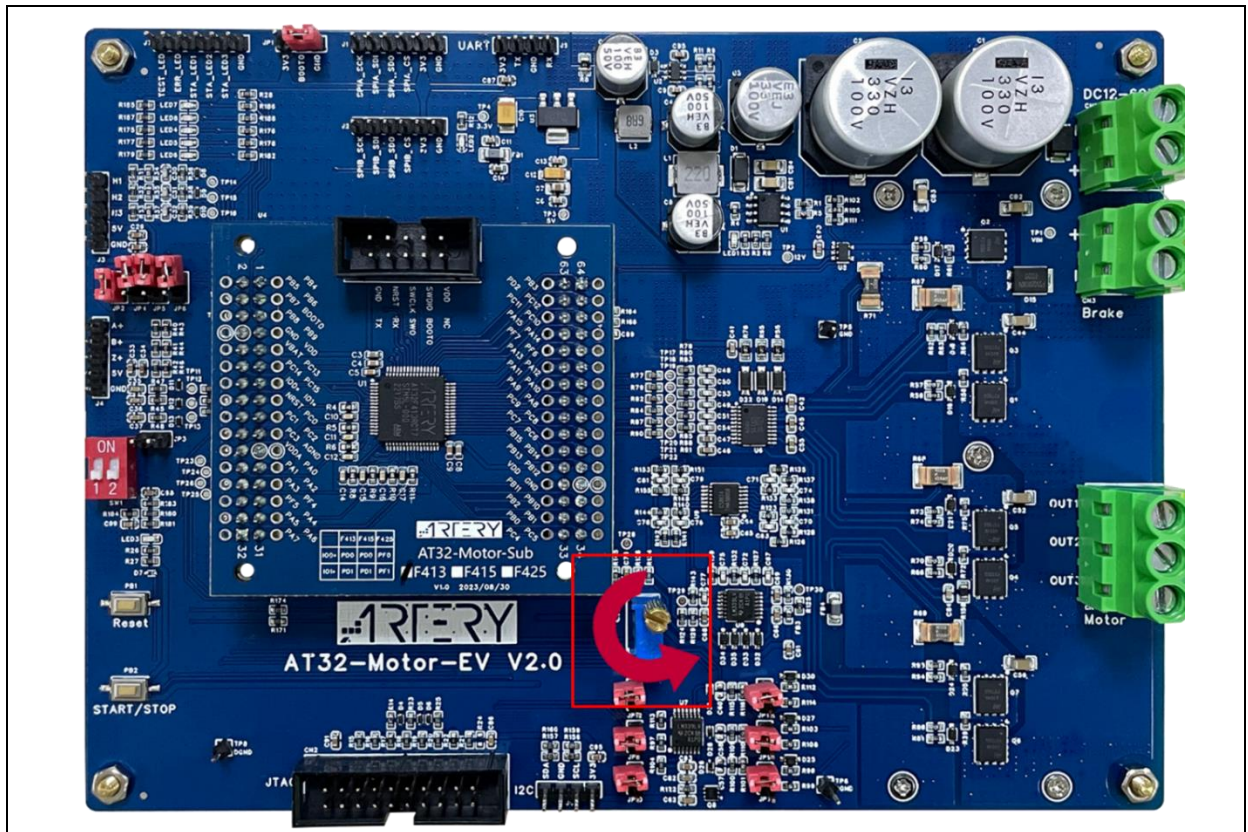
Figure 57. Parking lock flag set

Watch 1	
Name	Value
parking_lock_flag	0x01 SET
<Enter expression>	

3.19 External voltage control/software control mode

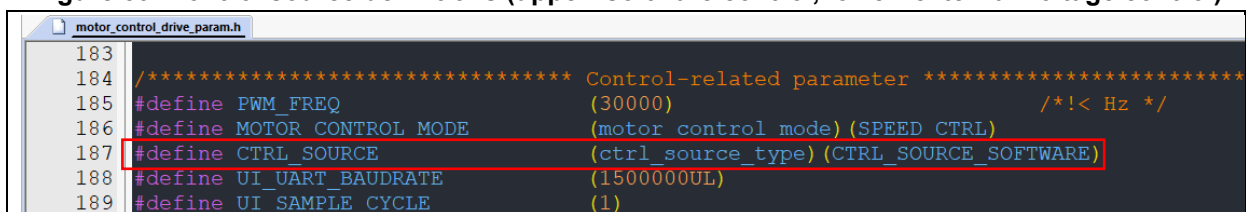
This demo project support two control sources, i.e., external voltage control and software control. The external voltage control mode adjusts speed or torque through external voltage. Users can adjust the potentiometer (as shown in the red box below) to control the command value. The voltage increases when the potentiometer is turned **counterclockwise**. The software control mode inputs speed or torque command value for control purpose. In this demo project, software control mode is set by default.

Figure 58. External voltage control – potentiometer



Users can select the control source by modifying the corresponding definition (CTRL_SOURCE in *motor_control_drive_param.h* file, as shown in Figure 59) or through PC software (see Figure 60). Control parameters are different for different control modes, such as the target speed (as shown in Figure 60) in speed control mode, or torque reference (as shown in Figure 61) in torque control mode.

Figure 59. Control source definitions (upper: software control, lower: external voltage control)



```

183
184 /***** Control-related parameter *****/
185 #define PWM_FREQ (30000) /*!< Hz */
186 #define MOTOR_CONTROL_MODE (motor_control_mode)(SPEED_CTRL)
187 #define CTRL_SOURCE (ctrl_source_type)(CTRL_SOURCE_EXTERNAL)
188 #define UI_UART_BAUDRATE (1500000UL)
189 #define UI_SAMPLE_CYCLE (1)

```

1) Select “External voltage control” as a control source

Select “external voltage control” in “Control source”. In this mode, it is possible to adjust speed or torque value by setting external voltage values. The “Target speed” and “Torque reference” area will generate the control speed or torque calculated based on the current voltage.

Note: The “Target speed” and “Torque reference” cannot be modified in “external voltage control” mode.

Figure 60. Speed control mode (external voltage control)

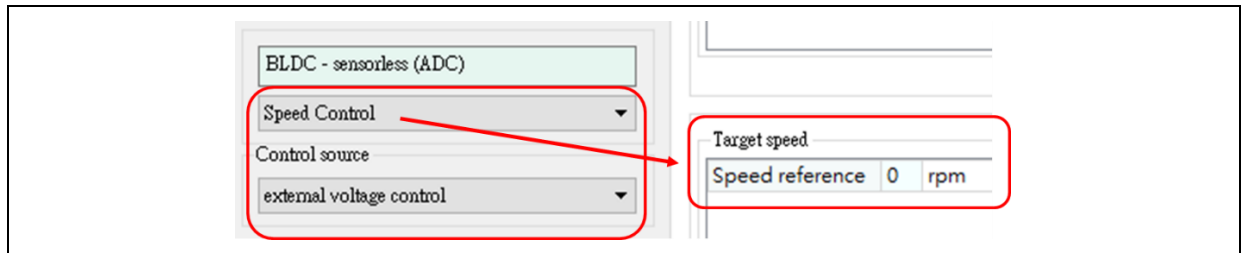
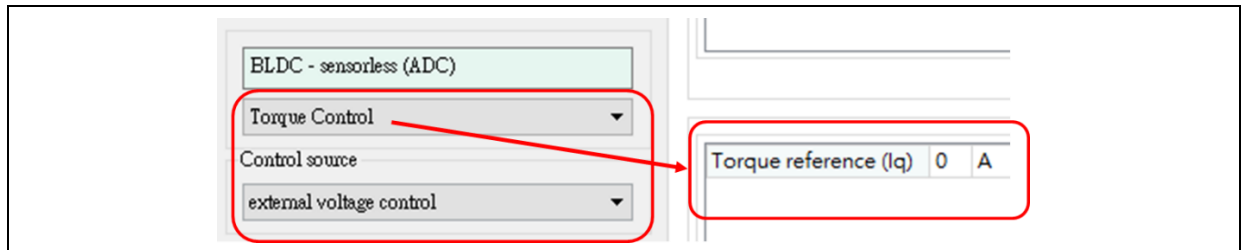


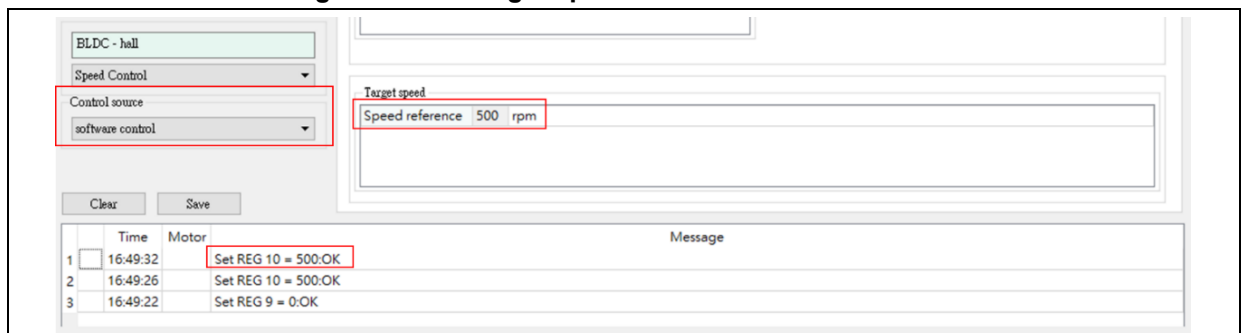
Figure 61. Torque control mode (external voltage control)



2) Select “Software control” as a control source

The software control mode is selected by default, as shown in Figure 62. In this mode, the user can tune the motor speed and torque by changing the target speed/torque reference. The “message” area will show whether the changing operation is successful or not.

Figure 62. Set target speed in software control mode



4 Revision history

Table 24. Document revision history

Date	Version	Revision note
2024.05.24	2.0.0	Initial release.
2024.12.27	2.1.1	1. Added AT32F435/437 series and AT32L021 series support; 2. Added description of IAR environment and *.icf file modification for Flash configuration; 3. Updated definitions, code and some descriptions.
2025.04.11	2.1.2	Added support for the AT32F425 series; Added new functionalities including speed control, field-weakening control, and MTPA/MTPV table generation and lookup features.
2025.10.09	2.1.3	Added parking/anti-theft modes, expanded supported models, corrected flash configuration tables, and updated parameters.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.

© 2025 ARTERY Technology – All Rights Reserved